Infosys Senior DevOps Engineer — Interview Preparation Handbook

Master Index

> ***Target Role:*** *Senior DevOps Engineer (Azure DevOps, Terraform, Kubernetes, CI/CD)* ***Target Company:*** *Infosys* ***Your Profile:*** *4 YOE, Linux/Jenkins/GitLab/Docker/K8s/AWS/OCI/Prometheus stack* ***Interview is:*** *TOMORROW*

Interview Round Structure at Infosys

Round	Duration	Focus	Weight
Technical Round 1	45–60 min	Core DevOps, Tools, Project Deep-Dive	40%
Technical Round 2	45–60 min	Architecture, Scenarios, Troubleshooting	30%
Managerial Round	30–45 min	Leadership, Problem-Solving, Client Handling	15%
HR Round	20–30 min	Fitment, Salary, Notice Period, Relocation	15%

Handbook Parts

Part 1 — Behavioral & Project Discussion

01_behavioral_project.md - Chapter 1: Tell Me About Yourself - Chapter 2: Current Project Discussion - Chapter 3: Daily Responsibilities - Chapter 4: Production Support - Chapter 5: Incident Handling

Part 2 — CI/CD & Azure DevOps

02_cicd_azure_devops.md - Chapter 6: CI/CD Fundamentals - Chapter 7: Azure DevOps Overview - Chapter 8: Azure Repos - Chapter 9: Azure Pipelines & YAML - Chapter 10: Azure Artifacts, Service Connections, Variable Groups - Chapter 11: Azure Key Vault, ACR, AKS (Azure-specific) - Chapter 12: Jenkins (Your Existing Tool)

Part 3 — Terraform Deep Dive

03_terraform.md - Chapter 13: Terraform Core Concepts & Interview Examples - Terraform Troubleshooting (50 Scenarios)

Part 4 — Docker Deep Dive

04_docker.md - Chapter 14: Docker Fundamentals, Dockerfile, Compose, Security

Part 5 — Kubernetes Deep Dive

05_kubernetes.md - Chapter 15: Kubernetes Concepts & Interview-Ready YAMLs - Chapter 16: Kubernetes Troubleshooting (100 Scenarios)

Part 6 — Linux Deep Dive

06_linux.md - Chapter 17: Linux — 100 Interview Questions, Commands, Troubleshooting

Part 7 — Git & Monitoring

07_git_monitoring_security.md - Chapter 18: Git - Chapter 19: Monitoring & Observability - Chapter 20: Security & DevSecOps

Part 8 — Architecture, HA, DR, Scalability

08_architecture.md - Chapter 21: Cloud Architecture - Chapter 22: High Availability - Chapter 23: Disaster Recovery - Chapter 24: Scalability - 50 Architecture Interview Questions

Part 9 — Scenarios, Consulting, Agile, Manager & HR

09_scenarios_hr.md - Chapter 25: DevOps Consulting Questions - Chapter 26: Agile/Scrum - Chapter 27: Scenario Questions (200+) - Chapter 28: Behavioral Questions (100 STAR) - Chapter 29: Manager Round - Chapter 30: HR Round - Chapter 31: Salary Discussion

Part 10 — Cheat Sheets, Revision & Mock Interviews

10_cheatsheets_revision.md - Chapter 32: One-Page Cheat Sheets (11 topics) - Last Night Revision (30 min / 1 hr / 3 hr / 6 hr) - Mock Interview Question Sets (Easy → Expert, 50 each)

Topic Probability Rankings

Based on Glassdoor, Medium, Reddit, and LinkedIn interview reports for Infosys DevOps roles:

Rank	Topic	Probability	Category
1	Tell me about yourself / Project discussion	🔴 **99%**	Very Frequently Asked
2	CI/CD pipeline design & troubleshooting	🔴 **95%**	Very Frequently Asked
3	Docker (Dockerfile, multi-stage, compose)	🔴 **95%**	Very Frequently Asked
4	Kubernetes (deployments, services, troubleshooting)	🔴 **90%**	Very Frequently Asked
5	Linux commands & troubleshooting	🔴 **90%**	Very Frequently Asked
6	Git (branching, merge, rebase, conflicts)	🔴 **90%**	Very Frequently Asked
7	Terraform (state, modules, plan/apply)	🟠 **85%**	Very Frequently Asked

Rank	Topic	Probability	Category
8	Azure DevOps (pipelines, repos, boards)	🟠 85%	Very Frequently Asked
9	Monitoring (Prometheus, Grafana, alerts)	🟠 80%	Frequently Asked
10	Production incident handling	🟠 80%	Frequently Asked
11	Cloud networking (VNet, subnets, NSG)	🟡 70%	Frequently Asked
12	Security (RBAC, secrets, DevSecOps)	🟡 70%	Frequently Asked
13	HA / DR / Scalability architecture	🟡 65%	Frequently Asked
14	Agile / Scrum methodology	🟡 60%	Sometimes Asked
15	Azure Key Vault / ACR / AKS specifics	🟡 55%	Sometimes Asked
16	Cost optimization	🔵 40%	Sometimes Asked
17	Ansible / Configuration management	🔵 35%	Sometimes Asked
18	Helm charts	🔵 30%	Sometimes Asked
19	Service mesh (Istio)	⚪ 15%	Rare but Possible
20	GitOps (ArgoCD, Flux)	⚪ 15%	Rare but Possible
21	Chaos engineering	⚪ 5%	Rare but Possible

Part 1 — Behavioral & Project Discussion

Chapter 1: Tell Me About Yourself

Probability: 🔴 99% — Asked in EVERY round

Why Interviewers Ask This

This is NOT a casual question. It is your **first impression audit**. The interviewer is evaluating: - Can you communicate clearly and concisely? - Do you understand your own career trajectory? - Are you a fit for this specific role? - Do you show confidence without arrogance?

The Formula (for 4 YOE DevOps)

```
[1] Hook (who you are) → 15 seconds
[2] Core experience (what you do) → 30 seconds
[3] Key achievements (proof of impact) → 30 seconds
[4] Why this role / why now → 15 seconds
```

Your Scripted Answer

> *"Hi, I'm Aniket. I'm a Senior DevOps Engineer with 4+ years of experience, currently working as a Technical Lead at Kochar Innovations, where I lead a DevOps team of 8+ engineers.*
>
> *My core expertise is in building and managing production infrastructure — CI/CD pipelines with Jenkins and GitLab CI, containerized workloads on Kubernetes and Docker, full-stack observability with Prometheus, Grafana, and Loki, and Infrastructure as Code with Terraform and Ansible.*
>
> *Some of my key contributions include containerizing 20+ microservices with a 60% image size reduction, architecting HA database platforms with Patroni and Redis Sentinel, driving cloud cost optimization that saved 25% monthly spend, and supporting ISO 27001 and SOC 2 compliance initiatives. I've managed 15+ production applications with 99.9% SLA.*
>
> *I'm now looking to take on a Senior DevOps role at Infosys because I want to work on larger-scale Azure and Kubernetes environments across multiple client projects, and I believe my infrastructure ownership and platform engineering experience directly aligns with what this role requires."*

Common Mistakes to AVOID

Mistake	Why It's Bad	Fix
Starting with "I was born in..."	Wastes time, unprofessional	Start with your current role
Listing every tool you've used	Sounds like reading a resume	Focus on 3-4 core strengths

Mistake	Why It's Bad	Fix
Being vague ("I do DevOps stuff")	Shows no depth	Use specifics: numbers, tools, impact
Speaking for 5+ minutes	Loses interviewer attention	Keep it under 90 seconds
Not mentioning the target role	Missed connection opportunity	End with why THIS role
Memorizing word-for-word	Sounds robotic	Know the structure, not the script

Follow-up Questions to Expect

1. **"Can you elaborate on your team lead responsibilities?"** → Emphasize: tech lead for 8+, direct management of 2-3 interns, sprint coordination, RCA ownership, incident coordination
2. **"Why are you leaving your current company?"** → "I'm looking for enterprise-scale projects with larger client exposure. Infosys offers the opportunity to work across multiple industries and cloud environments."
3. **"What's your biggest technical achievement?"** → Pick ONE. Go deep. Example: "Containerizing 20+ services with multi-stage Dockerfiles, reducing image sizes by 60% and deployment time by 50%, while implementing Grype-based security scanning that blocked 90% of CVE-exposed deployments."
4. **"What's your experience with Azure?"** → Be HONEST: "My production experience is primarily with AWS and OCI. However, I have strong conceptual understanding of Azure services — Azure DevOps maps to Jenkins+GitLab, AKS maps to my Kubernetes experience, Azure Monitor maps to Prometheus+Grafana. The underlying engineering principles are the same; the API surface is what changes."

Chapter 2: Current Project Discussion

Probability: 🔴 99% — Asked in EVERY technical round

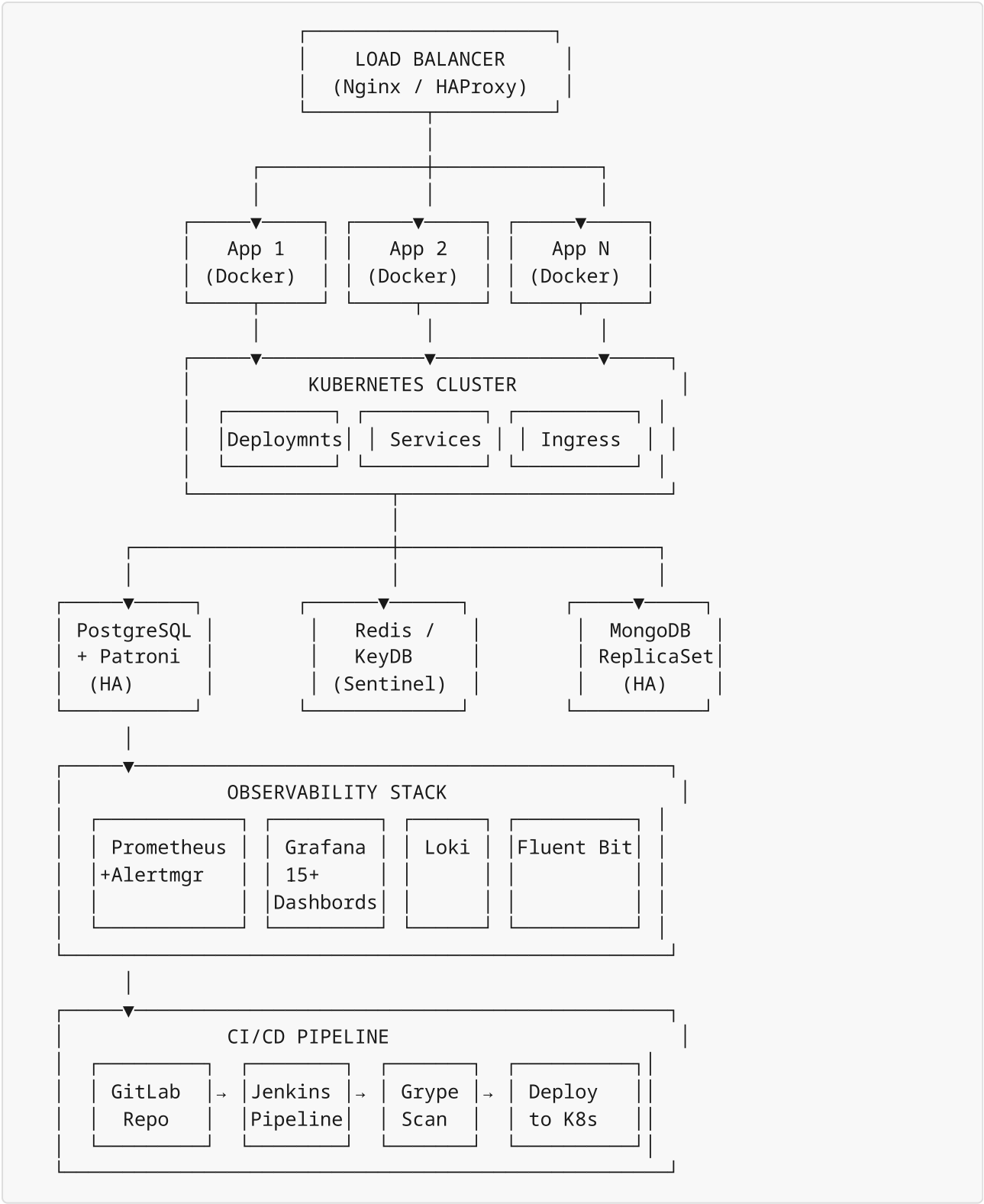
Why This Matters

At Infosys, interviewers **pick apart your current project** to assess: - How deeply you understand your own architecture - Whether you actually built it or just operate it - Your decision-making process - Your ability to explain complex systems simply

How to Structure Your Project Walkthrough

```
[1] Project context (what, who, why) → 30 seconds
[2] Architecture overview → 60 seconds
[3] Your specific contributions → 60 seconds
[4] Challenges & how you solved them → 60 seconds
[5] Impact / metrics → 30 seconds
```

Your Project Architecture (ASCII Diagram)



Sample Explanation

> *"I work at Kochar Innovations where we manage 15+ production applications across multi-cloud environments — primarily AWS and OCI, with some on-premise infrastructure.*
>
> *The architecture is microservices-based. We have 20+ containerized services running on Kubernetes, fronted by Nginx/HAProxy load balancers. Our data layer includes PostgreSQL with Patroni for automatic failover, MongoDB replica sets, Redis Sentinel for caching, and ClickHouse for analytics.*

> *For CI/CD, we use a combination of Jenkins and GitLab CI/CD. Code is pushed to GitLab, triggers a Jenkins pipeline that runs unit tests, builds multi-stage Docker images, runs Grype vulnerability scanning, and deploys to Kubernetes using rolling updates.*
>
> *For observability, I built the entire monitoring stack from scratch — Prometheus scrapes metrics from 100+ hosts and 200+ containers, Alertmanager routes alerts, Grafana has 15+ dashboards, and Loki+Fluent Bit handles centralized logging.*
>
> *My specific contributions include defining infrastructure standards and deployment patterns adopted across multiple application teams, implementing disaster recovery with automated snapshots and cross-region replication achieving sub-30-minute RTO, and driving cloud cost optimization that reduced spend by 25%."*

Deep-Dive Questions to Expect

Question	What They're Really Asking	How to Answer
"Why did you choose Patroni over pgpool?"	Do you understand HA tradeoffs?	"Patroni provides automatic leader election via etcd/consul, handles fencing properly, and integrates well with our existing tooling. pgpool is more of a connection pooler that also does HA, but Patroni is purpose-built for PostgreSQL HA."
"How do you handle database schema migrations?"	Do you think about data layer in CI/CD?	"We use migration tools (Flyway/Liquibase concepts) integrated into the deployment pipeline. Migrations run as a pre-deploy step, with backward-compatible changes first, then destructive changes in a separate release."
"What happens if your Jenkins master goes down?"	Do you design for resilience?	"We have Jenkins configured with periodic backup of JENKINS_HOME. For critical pipelines, we also have GitLab CI/CD as a fallback. Our infrastructure standards include documented recovery runbooks with tested RTO."
"How do you handle secrets?"	Security awareness	"Secrets are stored in vault/secret management, never in code. In K8s, we use Kubernetes Secrets (encrypted at rest). In CI/CD, credentials are managed through Jenkins credentials store with appropriate RBAC."
"Walk me through a deployment failure"	Incident response maturity	Use STAR format. Show you have a process, not just panic.

Common Mistakes

1. **Not knowing your own architecture** — If you say "PostgreSQL with Patroni" and can't explain how Patroni leader election works, you're in trouble
2. **Saying "the team did it"** — Use "I" for your contributions, "we" for team efforts. Own your work
3. **Not having metrics** — Always quantify. "We improved deployment speed" is weak. "Reduced deployment time from 45 minutes to 12 minutes" is strong
4. **Overcomplicating** — Don't try to make your project sound bigger than it is. Simple + deep > complex + shallow

Chapter 3: Daily Responsibilities

Probability: 🟠 80% — Frequently Asked

Why They Ask

They want to know if your daily work matches what they need. For a Senior DevOps role at Infosys, they need someone who can: - Handle production independently - Make infrastructure decisions - Coordinate across teams - Troubleshoot under pressure

How to Frame Your Daily Responsibilities

Morning Routine:

```
08:30 - Check monitoring dashboards (Grafana)
      - Review overnight alerts (Alertmanager/PagerDuty)
      - Check pipeline status (Jenkins/GitLab)
09:00 - Daily standup (Agile/Scrum)
      - Report blockers
      - Coordinate with dev teams
```

Core Work:

```
09:30-13:00 - Infrastructure work
           - Pipeline improvements
           - IaC changes (Terraform/Ansible)
           - Code reviews for infra changes
           - Deployment support
14:00-17:00 - Incident response (if any)
           - Sprint tasks
           - Documentation updates
           - Mentoring juniors/interns
           - Capacity planning
17:00-17:30 - End-of-day checks
           - Handoff to on-call if applicable
```

Interview Answer Template

> *"My typical day starts with checking our Grafana dashboards and Alertmanager for any overnight alerts. I review the CI/CD pipeline status in Jenkins and GitLab to catch any failed builds.*
>
> *At 9 AM, we have our daily Scrum standup where I coordinate with developers and QA. As the tech lead, I also do task assignment and sprint coordination for the DevOps team.*
>
> *The core of my day involves infrastructure work — writing Terraform code for infrastructure changes, improving CI/CD pipelines, handling deployment support, and doing code reviews for infrastructure changes. I also mentor 2-3 interns and provide technical guidance to the broader team.*

> *When incidents occur, I'm the primary coordinator — I handle triage, coordinate response, perform root cause analysis, and write post-incident reports. We follow ITIL-based incident management through Freshservice.*
>
> *I also dedicate time to infrastructure standards — defining deployment patterns, writing operational runbooks, and ensuring our practices are consistent across all application teams."*

Follow-up Questions

- **"How do you prioritize when you have a production incident AND a sprint deadline?"** → "Production always takes priority. I communicate the delay to stakeholders immediately, delegate sprint tasks where possible, resolve the incident, complete the RCA, and then re-plan the sprint work."
- **"How many deployments do you handle per day?"** → "We moved from bi-weekly releases to multiple daily deployments. On average, 3-5 deployments per day across all applications."

Chapter 4: Production Support

Probability: 🟠 80% — Frequently Asked

What Infosys Wants to Hear

Infosys is a consulting company. Their clients expect **24/7 production stability**. They need people who: - Can handle production on-call - Know how to triage and escalate - Understand SLAs and SLOs - Can write RCAs

Production Support Framework

```
INCIDENT SEVERITY MATRIX
┌──────────┬──────────────────────┬──────────┬──────────┐
│ Severity │ Definition           │ Response │ Resolve  │
├──────────┼──────────────────────┼──────────┼──────────┤
│ SEV-1    │ Complete outage      │ 15 min   │ 1 hour   │
│ SEV-2    │ Major degradation    │ 30 min   │ 4 hours  │
│ SEV-3    │ Minor issue          │ 2 hours  │ 24 hours │
│ SEV-4    │ Low impact           │ Next day │ 72 hours │
└──────────┴──────────────────────┴──────────┴──────────┘
```

Interview Questions

Q1: "How do you handle an on-call situation at 2 AM?"

> ***Answer:*** *"First, I check the alert details — severity, affected services, and impact. For a SEV-1, I immediately join the incident bridge, assess the blast radius, and start triage. My first step is always to check monitoring dashboards — Grafana for metrics, Loki for logs — to understand what changed. I follow the runbook if one exists. If the issue requires application-level fixes, I loop in the on-call*

> *developer. Throughout, I update the incident channel with status. After resolution, I document the timeline and write a blameless RCA within 24 hours."*

Q2: "Describe your SLA management approach"

> ***Answer:*** *"We define SLOs for each service — availability (99.9%), latency (p99 < 500ms), error rate (< 0.1%). These are monitored through Prometheus with Alertmanager rules. When we approach SLO burn rate thresholds, we get early warnings. This proactive approach helps us maintain our 99.9% SLA across 15+ applications. Monthly, we review SLO compliance reports in Grafana dashboards and adjust alert thresholds based on trends."*

Q3: "What's the difference between SLA, SLO, and SLI?"

Term	Definition	Your Example
SLI (Service Level Indicator)	The actual measured metric	Request latency p99 = 320ms
SLO (Service Level Objective)	The target you set internally	p99 latency should be < 500ms
SLA (Service Level Agreement)	The contractual promise to clients	99.9% uptime guaranteed

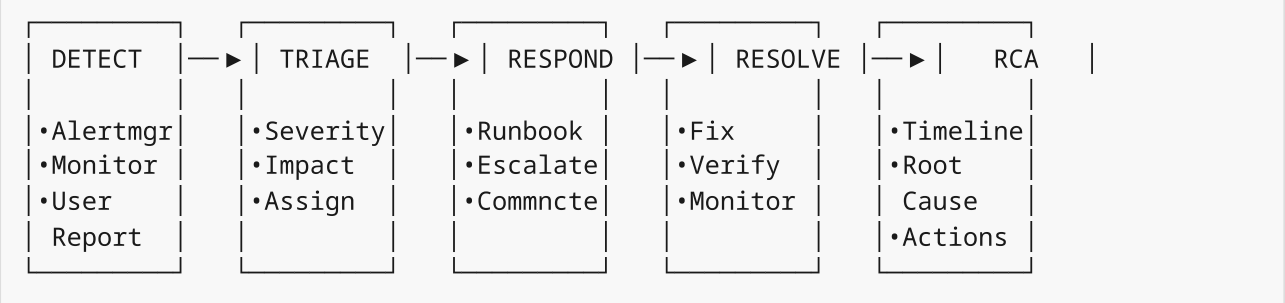
Production Tips from Real Experience

1. **Always have runbooks** — When you're debugging at 2 AM, you don't want to think. You want to follow steps.
2. **Pre-write incident response templates** — Slack/Teams message templates save 5 minutes of typing during crises
3. **Test your monitoring** — If Prometheus goes down, who monitors the monitor? Always have meta-monitoring.
4. **Backup your backups** — Cross-region replication. Test restores quarterly.

Chapter 5: Incident Handling

Probability: 🟠 80% — Frequently Asked (especially at consulting companies)

The Incident Lifecycle



STAR Format Example — Production Incident

Q: "Tell me about a critical production incident you handled."

> ***Situation:*** *"We had a PostgreSQL primary node failure during peak hours that affected 5 application services and 2,000+ active users."*
>
> ***Task:*** *"As the incident coordinator, I needed to restore database service, minimize downtime, and ensure data integrity."*
>
> ***Action:*** *"I immediately checked our Patroni cluster status and found the primary had OOM-killed due to a runaway query. Patroni had NOT automatically promoted the replica because the DCS (etcd) had a stale leader key. I manually triggered a controlled failover using `patronictl switchover`, verified data consistency by checking WAL replay status on the new primary, then identified and killed the runaway query. I updated the connection strings, drained the old primary, and restarted it as a replica."*
>
> ***Result:*** *"Total downtime was 8 minutes. We prevented data loss due to synchronous replication being enabled. I wrote an RCA that led to three improvements: 1) Added query timeout limits, 2) Configured OOM score adjustments for Patroni processes, 3) Updated our runbook with this specific failover scenario."*

RCA Template

```
# Root Cause Analysis — [Incident Title]
**Date:** YYYY-MM-DD
**Severity:** SEV-X
**Duration:** XX minutes
**Impact:** [Users/services affected]

## Timeline
| Time  | Event                              |
|-------|------------------------------------|
| 14:30 | Alert fired: PostgreSQL primary down |
| 14:32 | On-call acknowledged                |
| 14:35 | Triage: Patroni failover stuck      |
| 14:38 | Manual switchover initiated         |
| 14:40 | New primary accepting connections   |
| 14:42 | Application services recovered      |

## Root Cause
PostgreSQL OOM-killed due to unbounded analytical query.
Patroni auto-failover blocked by stale DCS leader key.

## Contributing Factors
1. No query timeout configured
2. No OOM score adjustment for critical processes
3. Runbook did not cover manual Patroni failover

## Action Items
| Action                   | Owner  | Deadline   | Status |
```

```
|--------------------------|--------|------------|--------|
| Configure statement_timeout| Aniket | 2025-04-01 | Done   |
| Set OOM score adj         | Aniket | 2025-04-01 | Done   |
| Update runbook            | Aniket | 2025-04-03 | Done   |

## Lessons Learned
- Synchronous replication prevented data loss
- Automated failover must be tested monthly
```

Common Interview Questions on Incident Handling

Question	Key Points to Cover
"How do you ensure incidents don't repeat?"	RCA → Action items → Runbook updates → Automated monitoring → Quarterly reviews
"What's your escalation process?"	Severity-based → On-call → Team lead → Manager → VP (if SEV-1 > 1 hour)
"How do you communicate during incidents?"	Dedicated channel, regular status updates (every 15 min for SEV-1), stakeholder notifications
"What tools do you use for incident management?"	Freshservice for ITSM, Slack/Teams for communication, Grafana/Prometheus for monitoring, Confluence/Wiki for RCA documentation
"What's a blameless post-mortem?"	Focus on SYSTEMS, not PEOPLE. Ask "what failed" not "who failed". Goal is learning and improvement, not blame.

Best Practices

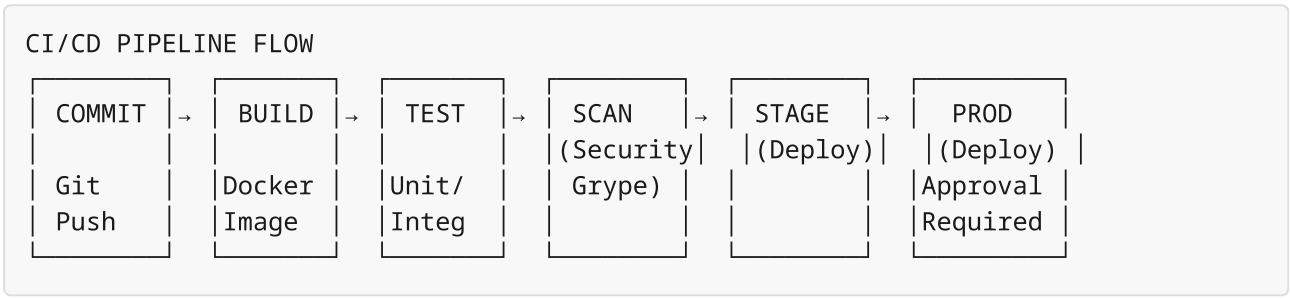
1. **Never deploy on Fridays** — Unless you have weekend on-call coverage
2. **Always have a rollback plan** — Before every deployment, know how to undo it
3. **Test your DR** — A DR plan that's never been tested is just a wish
4. **Automate toil** — If you're doing it manually more than twice, automate it
5. **Document everything** — Your future 2 AM self will thank you

Part 2 — CI/CD & Azure DevOps

Chapter 6: CI/CD Fundamentals

Probability: 🔴 95%

Core Concepts



Top Interview Questions

Q1: "What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?"

Term	Definition	Key Difference
CI	Automatically build + test on every commit	Merge to main frequently
Continuous Delivery	CI + automatically deploy to staging. Prod requires MANUAL approval	Human gate before prod
Continuous Deployment	CI + automatically deploy to ALL environments including prod	No human gate — fully automated

> ***Your answer:*** *"We practice Continuous Delivery. Every commit triggers build, test, and security scan. Staging deploys are automatic, but production requires manual approval through our change management process in Freshservice."*

Q2: "How do you handle rollbacks in your pipeline?"

> *"We use Kubernetes rolling updates with `maxUnavailable: 0` so old pods stay running until new ones pass readiness probes. For rollback, we use `kubectl rollout undo deployment/<name>`. Our Docker images are tagged with Git SHA, so we can always redeploy a known-good version. For database changes, we ensure backward compatibility — schema migrations are always additive first."*

Q3: "What is pipeline-as-code and why does it matter?"

> *"Pipeline-as-code means defining your CI/CD pipeline in a version-controlled file (Jenkinsfile, .gitlab-ci.yml, azure-pipelines.yml) rather than configuring through a UI. Benefits: versioned, reviewable, reproducible, auditable, and it enables GitOps practices."*

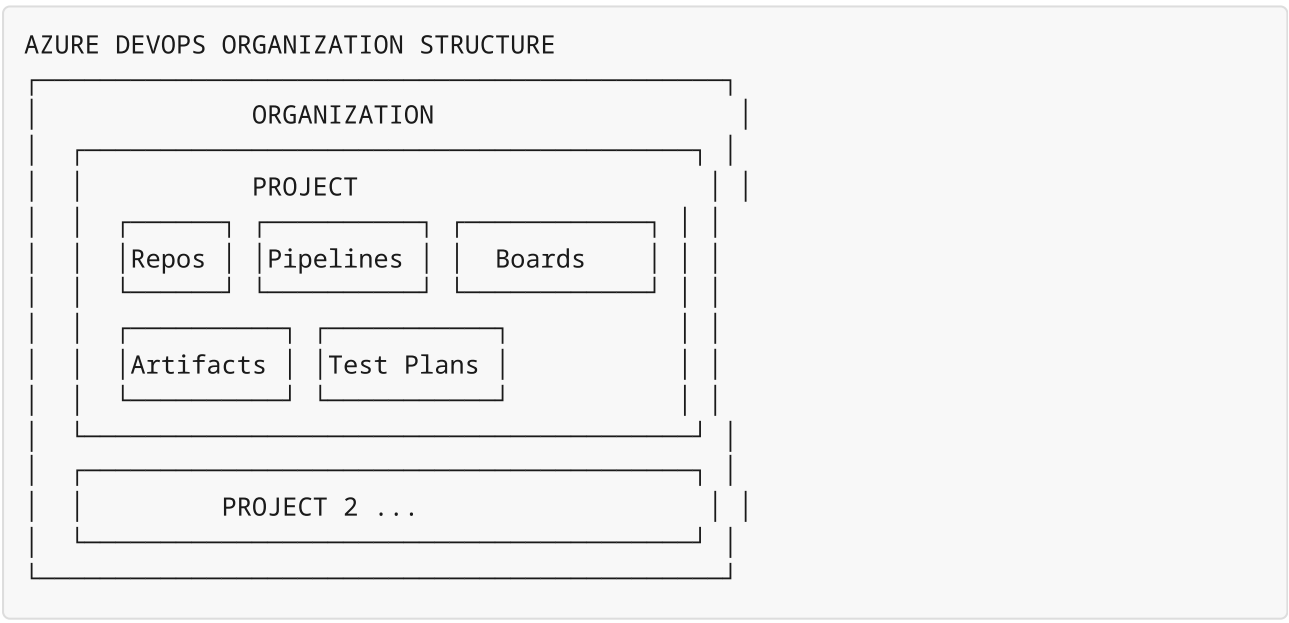
Chapter 7: Azure DevOps Overview

Probability: 🟠 85%

Mapping Azure DevOps to Your Experience

Azure DevOps Service	Your Equivalent	Key Differences
Azure Repos	Self-hosted GitLab	Git-based, similar. Azure Repos supports TFVC too
Azure Pipelines	Jenkins + GitLab CI	YAML-based, hosted agents, deeper Azure integration
Azure Boards	Jira	Work items, sprints, Kanban boards
Azure Artifacts	Nexus / JFrog Artifactory	npm, Maven, NuGet, pip, universal packages
Azure Test Plans	Manual QA tools	Exploratory + automated test management

Architecture



Key Interview Questions

Q: "What are the main components of Azure DevOps?"

> *"Azure DevOps is a suite of 5 services: Azure Repos for source control, Azure Pipelines for CI/CD, Azure Boards for project tracking, Azure Artifacts for package management, and Azure Test Plans for testing.*

> *These can be used independently or together. Think of it as GitLab's all-in-one approach — GitLab combines SCM, CI/CD, issues, and registry similarly."*

Q: "Azure DevOps vs GitHub Actions — when would you choose each?"

> *"Azure DevOps is better for enterprise environments needing boards, test plans, TFVC support, and complex multi-stage release pipelines with approvals. GitHub Actions is leaner, better for open-source and simpler workflows. For Infosys consulting projects, Azure DevOps is typically preferred because of its enterprise governance features."*

Q: "What's the difference between Classic and YAML pipelines?"

Feature	Classic (UI)	YAML
Definition	GUI-based, click-to-configure	Code-based, version-controlled
Version Control	Not tracked in Git	Lives in repo, full Git history
Reusability	Limited (task groups)	Templates, extends, parameters
Best For	Quick prototyping, legacy	Production, enterprise, audit
Recommendation	Avoid for new projects	**Always use YAML**

Chapter 8: Azure Repos

Probability: 🟡 55%

Mapping to GitLab

Feature	Azure Repos	GitLab	Notes
Git hosting	✅	✅	Functionally identical
Branch policies	✅ (UI-based)	✅ (Protected branches)	Azure: require reviewers, linked work items, successful build
Pull Requests	✅	✅ (Merge Requests)	Same concept, different name
Code search	✅	✅	
TFVC support	✅	❌	Legacy. Never recommend TFVC
Fork workflow	✅	✅	

Key Interview Questions

Q: "How do you enforce code quality in Azure Repos?"

> *"Through Branch Policies on the main branch: 1) Require minimum number of reviewers (usually 2), 2) Require linked work item for traceability, 3) Require successful build validation (pipeline must pass before merge), 4) Require comment resolution, 5) Optionally require specific reviewers for certain paths (CODEOWNERS equivalent)."*

Q: "What branching strategy do you recommend?"

> *"For most teams, I recommend trunk-based development with short-lived feature branches. Feature branches should live no more than 1-2 days. This reduces merge conflicts and enables CI. For larger teams or regulated environments, GitFlow with develop/release branches may be appropriate, but it adds complexity."*

Chapter 9: Azure Pipelines & YAML

Probability: 🟠 85%

Pipeline Structure

```yaml
# azure-pipelines.yml — Complete Example
trigger:                          # WHEN to run
  branches:
    include: [main, develop]
  paths:
    exclude: ['docs/*']

pool:                              # WHERE to run
  vmImage: 'ubuntu-latest'        # Microsoft-hosted agent
  # OR
  # name: 'SelfHostedPool'        # Self-hosted agent

variables:                         # Configuration
  dockerRegistry: 'myacr.azurecr.io'
  imageName: 'myapp'
  tag: '$(Build.BuildId)'

stages:                            # Pipeline stages
- stage: Build
  displayName: 'Build & Test'
  jobs:
  - job: BuildJob
    steps:
    - task: Docker@2               # Build Docker image
      inputs:
        command: 'build'
        repository: '$(imageName)'
        dockerfile: 'Dockerfile'
```

```
        tags: '$(tag)'

    - script: |                    # Run tests
        docker run --rm $(imageName):$(tag) npm test
      displayName: 'Run Unit Tests'

- stage: Deploy_Staging
  displayName: 'Deploy to Staging'
  dependsOn: Build
  condition: succeeded()
  jobs:
  - deployment: DeployStaging      # deployment job (not regular job)
    environment: 'staging'         # links to Azure DevOps Environment
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs:
              action: 'deploy'
              manifests: 'k8s/staging/*.yml'

- stage: Deploy_Production
  displayName: 'Deploy to Production'
  dependsOn: Deploy_Staging
  condition: succeeded()
  jobs:
  - deployment: DeployProd
    environment: 'production'      # Has manual approval gate
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@0
            inputs:
              action: 'deploy'
              manifests: 'k8s/prod/*.yml'
```

Jenkins vs Azure Pipelines — Side by Side

```
JENKINS (Jenkinsfile)                AZURE PIPELINES (YAML)
─────────────────────                ──────────────────────
pipeline {                           trigger:
  agent any                            branches: [main]
                                     pool:
                                       vmImage: 'ubuntu-latest'
  stages {                           stages:
    stage('Build') {                 - stage: Build
      steps {                          jobs:
        sh 'docker build .'           - job: BuildJob
      }                                  steps:
    }                                    - script: docker build .
    stage('Deploy') {                - stage: Deploy
      steps {                          jobs:
        sh 'kubectl apply ...'         - deployment: DeployJob
```

```
      }                                  environment: 'prod'
    }
  }
  post {                              # No direct equivalent
    failure {                         # Use conditions:
      mail to: 'team@...'            # condition: failed()
    }
  }
}
```

Key Concepts Mapping

Jenkins	Azure Pipelines	Notes
Jenkinsfile	azure-pipelines.yml	Both are pipeline-as-code
Agent/Node	Pool/Agent	Azure has Microsoft-hosted + self-hosted
Stage	Stage	Same concept
Step	Step/Task	Azure has pre-built Tasks from marketplace
Shared Library	Templates	Azure uses `extends` and `template` keywords
Credentials	Service Connections	Azure integrates with Key Vault
Post actions	Conditions	`condition: always()`, `condition: failed()`
Multibranch	Trigger with branch filters	Similar behavior

Top Interview Questions

Q: "What is the difference between a Microsoft-hosted and self-hosted agent?"

Feature	Microsoft-Hosted	Self-Hosted
Provisioning	Auto — fresh VM each run	You manage the VM/container
Cost	Included (limited free mins)	Your infra cost
Customization	Limited to pre-installed tools	Full control
Network	Public internet	Can access private networks
Security	Managed by Microsoft	Your responsibility
When to use	Simple builds, public repos	Private networks, custom tools, compliance

> ***Your answer:*** *"This is similar to Jenkins. Microsoft-hosted agents are like using a cloud-based Jenkins agent — fresh environment each time, no maintenance. Self-hosted agents are like our on-premise Jenkins agents — we control the tooling, they can access internal networks, and we handle patching. I'd use self-hosted when the pipeline needs to access private resources like internal registries or databases."*

Q: "Explain stages, jobs, steps, and tasks"

```
Pipeline
├── Stage 1 (Build)           ← Logical boundary, can have approvals
│   ├── Job 1 (Compile)       ← Runs on ONE agent, parallel with other jobs
│   │   ├── Step 1 (Checkout) ← Individual action
│   │   ├── Step 2 (Build)
│   │   └── Step 3 (Test)
│   └── Job 2 (Lint)          ← Runs in parallel with Job 1
│       └── Step 1 (Run ESLint)
├── Stage 2 (Deploy Staging)  ← Depends on Stage 1
│   └── Deployment Job        ← Special job type with environment + strategy
└── Stage 3 (Deploy Prod)     ← Manual approval before this stage
    └── Deployment Job
```

Q: "How do you implement approvals in Azure Pipelines?"

> *"Approvals are configured on Environments, not on the pipeline itself. You create an Environment (e.g., 'production') in Azure DevOps, then add Approval checks — requiring specific people or groups to approve before any deployment job targeting that environment can proceed. This is like Jenkins' `input` step but more structured and auditable. You can also add business hours checks, exclusive locks, and branch control."*

Chapter 10: Artifacts, Service Connections, Variable Groups

Service Connections (Probability: 🟠 70%)

> ***Jenkins equivalent:*** *Credentials (stored in Jenkins Credential Store)*

Q: "What is a Service Connection and how do you secure it?"

> *"A Service Connection is how Azure Pipelines authenticates to external services — Azure subscriptions, Docker registries, Kubernetes clusters, AWS, etc. It's like a Jenkins credential but with RBAC at the project level. Best practices: 1) Use Workload Identity Federation (OIDC) instead of client secrets — no secrets to rotate. 2) Scope to specific resource groups using least-privilege. 3) Restrict which pipelines can use it. 4) Audit usage regularly."*

Variable Groups (Probability: 🟡 60%)

> ***Jenkins equivalent:*** *Environment variables + Credentials combined*

```
# Referencing a variable group in YAML
variables:
- group: 'production-secrets'     # Linked from Azure DevOps Library
- name: buildConfiguration
  value: 'Release'

# Variable group can be linked to Azure Key Vault
# Secrets are fetched at runtime — never stored in pipeline
```

Q: "How do you manage environment-specific configuration?"

> *"I use Variable Groups — one per environment (dev, staging, prod). Sensitive values are linked to Azure Key Vault so secrets are fetched at runtime and never stored in the pipeline definition. Non-sensitive config (like API URLs, feature flags) goes directly in the variable group. This is cleaner than Jenkins where you'd typically manage this through a combination of credential stores, environment variables, and parameterized builds."*

Azure Artifacts (Probability: 🟡 40%)

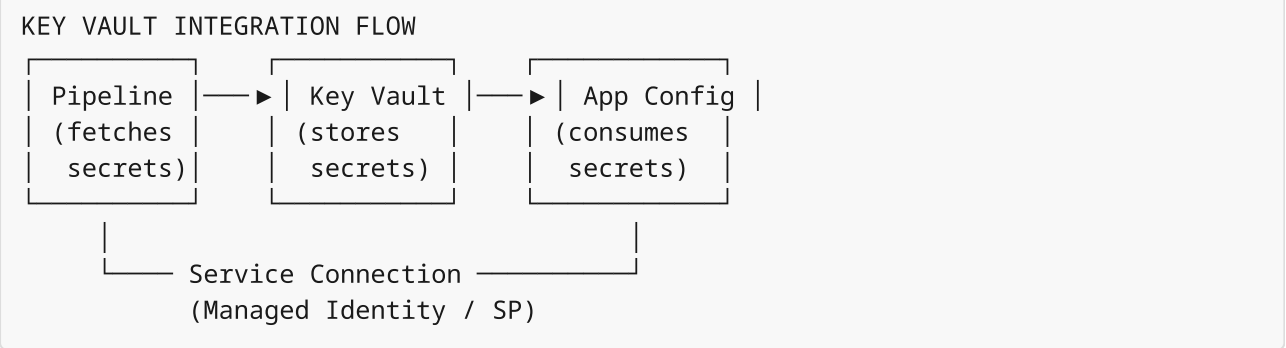
> ***Jenkins equivalent:*** *Nexus/Artifactory*

Q: "What is Azure Artifacts?"

> *"It's a package management service — hosts npm, Maven, NuGet, pip, and universal packages. Think of it as a private Nexus/Artifactory built into Azure DevOps. Key use: host internal packages that your pipelines consume. Supports upstream sources (proxy to public registries). Useful for caching public packages to avoid rate limiting and for hosting internal shared libraries."*

Chapter 11: Azure Key Vault, ACR, AKS

Azure Key Vault (Probability: 🟡 55%)



Q: "How does Azure Key Vault compare to what you use now?"

> *"Currently we use Kubernetes Secrets (encrypted at rest) and Jenkins Credential Store. Key Vault is more enterprise-grade — it provides centralized secret management with audit logging, automatic rotation, soft-delete protection, RBAC, and integration with Azure services. The equivalent in AWS is AWS Secrets Manager or SSM Parameter Store. In our setup, I'd link Key Vault to Variable Groups so pipelines fetch secrets at runtime without them ever being stored in pipeline code."*

Azure Container Registry (ACR) (Probability: 🟡 50%)

> ***Your equivalent:*** *Self-hosted GitLab Container Registry or DockerHub*

Q: "What's ACR and how would you integrate it?"

> *"ACR is Azure's private Docker registry — equivalent to our self-hosted GitLab registry. Key advantages: geo-replication for multi-region deployments, integrated vulnerability scanning with Microsoft Defender, content trust (image signing), and native integration with AKS (no ImagePullSecrets needed if using managed identity). In a pipeline: build image → push to ACR → AKS pulls from ACR using managed identity."*

Azure Kubernetes Service (AKS) (Probability: 🟡 55%)

> ***Your equivalent:*** *Self-managed Kubernetes clusters*

Q: "AKS vs self-managed Kubernetes — what's different?"

Feature	Self-Managed K8s	AKS
Control plane	You manage (etcd, API server, scheduler)	Azure manages (free)
Node management	You provision, patch, scale	Node pools, auto-scale, auto-upgrade
Networking	You configure CNI	Azure CNI or Kubenet built-in
Monitoring	Prometheus + Grafana (you build)	Azure Monitor + Container Insights
Security	You manage RBAC, policies	AAD integration, Azure Policy
Cost	Infra only	Infra only (control plane free)

> ***Your answer:*** *"My Kubernetes fundamentals are identical — deployments, services, ingress, RBAC, HPA, PVCs — the K8s API is the same. The difference is operational: AKS abstracts the control plane. I don't manage etcd backups or API server certificates. AKS adds Azure-specific features like AAD-integrated*

> *RBAC, Azure CNI networking, and node pool auto-scaling. My Prometheus+Grafana stack maps to Azure Monitor + Container Insights."*

Chapter 12: Jenkins (Your Existing Tool)

Probability: 🟠 80%

Declarative Pipeline

```
pipeline {
    agent {
        docker {
            image 'node:18-alpine'    // Run in container
            args '-v /tmp:/tmp'
        }
    }

    environment {
        REGISTRY = 'registry.example.com'
        IMAGE = "${REGISTRY}/myapp:${BUILD_NUMBER}"
    }

    options {
        timeout(time: 30, unit: 'MINUTES')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh 'npm ci'
                sh 'npm run build'
            }
        }

        stage('Test') {
            parallel {                  // Parallel execution
                stage('Unit Tests') {
                    steps { sh 'npm test' }
                }
                stage('Lint') {
                    steps { sh 'npm run lint' }
                }
            }
```

```
        }

        stage('Security Scan') {
            steps {
                sh "grype ${IMAGE} --fail-on critical"
            }
        }

        stage('Deploy') {
            when {
                branch 'main'           // Only deploy from main
            }
            input {                      // Manual approval
                message 'Deploy to production?'
                ok 'Yes, deploy'
            }
            steps {
                sh "kubectl apply -f k8s/"
            }
        }
    }

    post {
        failure {
            slackSend channel: '#alerts',
                message: "Build FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
        }
        always {
            cleanWs()                   // Clean workspace
        }
    }
}
```

Key Interview Questions

Q: "Declarative vs Scripted Pipeline?"

Feature	Declarative	Scripted
Syntax	`pipeline { }` block	`node { }` block
Structure	Predefined structure	Groovy flexibility
Error handling	`post { }` block	`try/catch/finally`
Validation	Pre-validated before run	Runtime errors possible
Recommendation	**Use for most cases**	Complex logic only

Q: "How do you handle Jenkins high availability?"

> *"Jenkins itself doesn't support active-active HA natively. Our approach: 1) Regular automated backups of JENKINS_HOME to S3/Object Storage, 2) Configuration-as-code plugin to rebuild from code, 3)*

> *Separate build agents from the master — agents are ephemeral Docker containers, 4) For critical pipelines, we have GitLab CI as a failover option."*

Q: "How do you manage Jenkins credentials securely?"

> *"We use the Jenkins Credential Store with folder-level scoping. Secrets are never printed in logs (masked automatically). For cloud access, we use IAM roles attached to the Jenkins agent instances rather than storing access keys. We rotate credentials quarterly and audit credential usage through the audit trail plugin."*

Jenkins Troubleshooting Scenarios

Scenario	Diagnosis	Fix
Build stuck, no logs	Agent disconnected	Check agent connectivity, restart agent
"No space left on device"	Workspace not cleaned	Add `cleanWs()` in post, increase disk, prune old builds
Pipeline hangs at `input`	Waiting for approval	Approve or add timeout: `timeout(time: 1, unit: 'HOURS')`
Plugin compatibility issue	After Jenkins/plugin update	Pin plugin versions, test upgrades in staging first
Concurrent build conflicts	Shared resources	Use `disableConcurrentBuilds()` or resource locks
Slow builds	Large workspace, no caching	Use pipeline caching, docker layer caching, parallel stages

Part 3 — Terraform Deep Dive

Probability: 🟠 85% — Very Frequently Asked

Core Concepts

Provider Configuration

```
# providers.tf
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"            # Pessimistic constraint: >=3.0, <4.0
    }
  }

  # Remote backend for team collaboration
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstateinfosys"
    container_name       = "tfstate"
    key                  = "prod.terraform.tfstate"
  }
}

provider "azurerm" {
  features {}                        # Required block, even if empty
  subscription_id = var.subscription_id
}
```

Q: "What does `~> 3.0` mean?"

> *"It's a pessimistic version constraint — allows 3.x.x but not 4.0.0. Protects against breaking changes while allowing patches. Similar to `^3.0.0` in npm."*

Variables

```
# variables.tf
variable "environment" {
  type        = string
  description = "Environment name (dev, staging, prod)"
  default     = "dev"
```

```
  validation {                        # Input validation
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "vm_size" {
  type    = string
  default = "Standard_B2s"
}

variable "tags" {
  type = map(string)
  default = {
    managed_by = "terraform"
    team       = "devops"
  }
}

variable "db_password" {
  type      = string
  sensitive = true                    # Masked in output and logs
}
```

Q: "How do you manage sensitive variables?"

> *"Mark them `sensitive = true` — Terraform masks them in plan/apply output. Never store them in `.tfvars` files committed to Git. Options: 1) Environment variables (`TF_VAR_db_password`), 2) Azure Key Vault data source, 3) Terraform Cloud variables marked sensitive, 4) CI/CD pipeline secret variables injected at runtime."*

Outputs & Locals

```
# outputs.tf
output "aks_cluster_name" {
  value       = azurerm_kubernetes_cluster.main.name
  description = "Name of the AKS cluster"
}

output "db_connection_string" {
  value     = azurerm_postgresql_server.main.fqdn
  sensitive = true                    # Sensitive output
}

# locals.tf
locals {
  name_prefix = "${var.project}-${var.environment}"
  common_tags = merge(var.tags, {
    environment = var.environment
    project     = var.project
```

```
  })
}
```

Azure Resource Examples

Resource Group + VNet + Subnet + NSG

```
resource "azurerm_resource_group" "main" {
  name     = "${local.name_prefix}-rg"
  location = var.location
  tags     = local.common_tags
}

resource "azurerm_virtual_network" "main" {
  name                = "${local.name_prefix}-vnet"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  address_space       = ["10.0.0.0/16"]
  tags                = local.common_tags
}

resource "azurerm_subnet" "app" {
  name                 = "app-subnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_network_security_group" "app" {
  name                = "${local.name_prefix}-app-nsg"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  security_rule {
    name                       = "AllowHTTPS"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "443"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name                       = "DenyAll"
    priority                   = 4096
    direction                  = "Inbound"
    access                     = "Deny"
    protocol                   = "*"
    source_port_range          = "*"
    destination_port_range     = "*"
```

```
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}
```

AKS Cluster

```
resource "azurerm_kubernetes_cluster" "main" {
  name                = "${local.name_prefix}-aks"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = local.name_prefix
  kubernetes_version  = "1.28"

  default_node_pool {
    name                = "system"
    node_count          = 3
    vm_size             = "Standard_D4s_v3"
    vnet_subnet_id      = azurerm_subnet.app.id
    enable_auto_scaling = true
    min_count           = 2
    max_count           = 5
    os_disk_size_gb     = 50
  }

  identity {
    type = "SystemAssigned"          # Managed Identity — no secrets to manage
  }

  network_profile {
    network_plugin = "azure"         # Azure CNI for VNet integration
    service_cidr   = "10.1.0.0/16"
    dns_service_ip = "10.1.0.10"
  }

  tags = local.common_tags
}
```

Modules

```
# modules/aks/main.tf — Reusable module
variable "cluster_name" { type = string }
variable "node_count"   { type = number; default = 3 }
variable "vm_size"      { type = string; default = "Standard_D4s_v3" }

resource "azurerm_kubernetes_cluster" "this" {
  name                = var.cluster_name
  # ... (full config)
}

output "kube_config" {
  value     = azurerm_kubernetes_cluster.this.kube_config_raw
  sensitive = true
}
```

```
# Root module usage
module "aks_prod" {
  source       = "./modules/aks"     # Local module
  # source     = "git::https://github.com/org/tf-modules.git//aks?ref=v2.1.0"  # Ve
  cluster_name = "prod-aks"
  node_count   = 5
  vm_size      = "Standard_D8s_v3"
}
```

Advanced Concepts

count vs for_each

```
# count — use for identical resources
resource "azurerm_subnet" "workers" {
  count                = 3
  name                 = "worker-subnet-${count.index}"
  address_prefixes     = ["10.0.${count.index + 10}.0/24"]
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
}

# for_each — use for distinct resources (PREFERRED)
variable "environments" {
  default = {
    dev     = { size = "Standard_B2s", count = 1 }
    staging = { size = "Standard_D2s_v3", count = 2 }
    prod    = { size = "Standard_D4s_v3", count = 3 }
  }
}

resource "azurerm_kubernetes_cluster" "env" {
  for_each            = var.environments
  name                = "${each.key}-aks"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix          = each.key

  default_node_pool {
    name       = "default"
    node_count = each.value.count
    vm_size    = each.value.size
  }
  identity { type = "SystemAssigned" }
}
```

Q: "When do you use count vs for_each?"

> *"Use `for_each` when resources are distinct (different configs, names). Use `count` only for truly identical resources. Key difference: `count` uses index — if you remove item at index 1, all subsequent*

> *items shift and get recreated. `for_each` uses map keys — removing one item doesn't affect others. `for_each` is almost always the safer choice."*

Lifecycle & depends_on

```
resource "azurerm_kubernetes_cluster" "main" {
  # ...

  lifecycle {
    create_before_destroy = true     # Zero-downtime replacement
    prevent_destroy       = true     # Prevent accidental deletion
    ignore_changes        = [tags]   # Ignore external tag changes
  }
}

# Explicit dependency (when Terraform can't infer)
resource "azurerm_role_assignment" "aks_acr" {
  principal_id         = azurerm_kubernetes_cluster.main.kubelet_identity[0].object
  role_definition_name = "AcrPull"
  scope                = azurerm_container_registry.main.id

  depends_on = [azurerm_kubernetes_cluster.main]
}
```

State Management

Q: "What happens if two people run `terraform apply` at the same time?"

> *"Without state locking, you get state corruption. That's why we use remote backends with locking: Azure Blob Storage uses blob leasing for locks (no DynamoDB needed like AWS). When one person runs apply, the state file is locked. Others see 'Error: Error locking state' and must wait. If a lock is stuck due to a crashed process, you can use `terraform force-unlock <LOCK_ID>` — but ONLY after confirming no apply is actually running."*

Q: "How do you recover from a corrupted state file?"

> *"1) If using remote backend with versioning enabled — restore previous version from blob storage. 2) If no backup — use `terraform import` to reconstruct state from existing infrastructure, resource by resource. 3) Prevention: always enable versioning on your state storage, run regular state backups, and use CI/CD for applies (never local)."*

Workspaces

```
terraform workspace new dev
terraform workspace new staging
```

```
terraform workspace new prod
terraform workspace select prod
terraform workspace list
```

Q: "Workspaces vs separate directories for environments?"

> *"I prefer separate directories (or separate root modules) with a shared module. Workspaces share the same code and backend — you can accidentally apply dev config to prod if you forget which workspace you're in. Separate directories with remote state make the blast radius explicit. Workspaces are fine for ephemeral environments (PR previews, testing)."*

Commands Cheat Sheet

Command	Purpose	When to Use
`terraform init`	Initialize, download providers/modules	First time, after adding providers
`terraform plan`	Preview changes	ALWAYS before apply
`terraform apply`	Execute changes	After reviewing plan
`terraform destroy`	Delete all resources	Teardown environments
`terraform fmt`	Format code	Before every commit
`terraform validate`	Syntax/logic validation	In CI pipeline
`terraform import`	Import existing resources	Brownfield adoption
`terraform state list`	List tracked resources	Debugging
`terraform state show`	Show resource details	Debugging
`terraform state mv`	Move/rename in state	Refactoring
`terraform state rm`	Remove from state (not infra)	Unmanage a resource
`terraform output`	Show outputs	Get values for scripts
`terraform taint` (deprecated)	Mark for recreation	Use `-replace` flag instead
`terraform plan -replace=RES`	Force recreate specific resource	Replace unhealthy resource

---

Terraform Troubleshooting — 50 Scenarios

#	Scenario	Root Cause	Fix
1	`Error: Error locking state`	Another apply is running or lock stuck	Wait, or `terraform force-unlock <ID>` after verification
2	State file corrupted	Concurrent apply without locking	Restore from versioned backend backup
3	`Error: Provider not found`	Missing `terraform init`	Run `terraform init`
4	`Error: Unsupported attribute`	Wrong provider version	Check docs, pin correct version
5	Resource recreated unexpectedly	Change in an immutable attribute (e.g., name)	Use `lifecycle { create_before_destroy = true }`
6	Drift detected in plan	Manual changes in console	`terraform apply` to reconcile, or update code to match
7	`Error: Cycle detected`	Circular dependency	Refactor — break cycle with intermediate resources or data sources
8	Plan shows destroying resources after `count` change	Index shift	Migrate to `for_each`
9	`terraform import` fails	Wrong resource address or ID format	Check provider docs for exact import ID format
10	Module source changed but not updating	Cached locally	`terraform init -upgrade`
11	Sensitive value shown in plan	Missing `sensitive = true`	Add `sensitive = true` to variable/output
12	`Error: creating X: StatusCode=403`	Insufficient permissions on Service Principal	Check RBAC, assign required roles
13	`Error: A resource with the ID already exists`	Resource exists but not in state	`terraform import` to bring into state
14	Apply times out	Long-running resource creation (AKS ~10-15 min)	Increase timeout in resource block
15	`Error: Invalid count argument`	Count depends on computed value	Use `depends_on` or restructure
16	State too large, slow plans	Monolithic state file	Split into smaller state files by component
17	`Error: Backend configuration changed`	Backend block modified	`terraform init -reconfigure` or `-migrate-state`
18	Destroy hangs on a resource	Resource has dependencies in cloud	Delete manually, then `terraform state rm`

#	Scenario	Root Cause	Fix
19	`Error: Module not installed`	Missing init after adding module	`terraform init`
20	Variable validation fails	Input doesn't match constraint	Check validation block conditions
21	Output shows `<sensitive>`	Sensitive flag set	Use `terraform output -json` to see values
22	`Error: creating NSG Rule: Rule with priority X exists`	Duplicate priority	Use unique priority numbers per NSG
23	Subnet conflicts with existing CIDR	Address space overlap	Choose non-overlapping CIDR ranges
24	AKS node pool won't scale	Quota limit reached	Request quota increase in Azure portal
25	`moved` block not working	Wrong `from` address	Use `terraform state list` to verify exact address
26	Provisioner fails	Network/SSH issue to target	Use `null_resource` with proper connection block
27	Remote exec timeout	SSH not reachable	Check NSG rules, public IP, SSH key
28	State shows resource but it's deleted manually	Orphaned state entry	`terraform state rm` the orphan, then `terraform apply`
29	Plan shows no changes but infra is wrong	State and infra in sync but code is wrong	`terraform refresh` (now part of plan), review carefully
30	`for_each` with list fails	`for_each` requires map or set	Use `toset()` or convert list to map
31	Dynamic block generates wrong config	Incorrect iterator	Check `content` block references
32	Terraform Cloud run queue stuck	Previous run pending	Discard pending run in TFC UI
33	`Error: Provider produced inconsistent result`	Provider bug or API race condition	Re-run apply, or report to provider
34	Can't delete resource group (has locks)	Management lock exists	Remove lock first, then destroy
35	Key Vault soft-delete conflict	Deleted vault with same name exists	Purge deleted vault first
36	AKS upgrade fails mid-way	Node pool version incompatibility	Can't skip versions — upgrade incrementally
37	`Error: retrieving X: unexpected status 404`	Resource deleted outside Terraform	`terraform state rm`, then re-apply
38	Large number of API calls hitting rate limits	Too many resources in one apply	Split state, use `-parallelism=5` flag

#	Scenario	Root Cause	Fix
39	Conditional resource creation unclear	Using `count = var.enabled ? 1 : 0`	Standard pattern — document clearly
40	Module version pinning not working	Using branch ref instead of tag	Use `?ref=v1.2.3` (tags, not branches)
41	`terraform plan` takes 10+ minutes	Large state, many resources	Split state, use targeted plans: `-target`
42	Accidental `terraform destroy` on prod	No safety net	Use `prevent_destroy = true` on critical resources
43	Environment variable not picked up	Wrong naming	Must be `TF_VAR_<name>`, case-sensitive
44	Data source returns empty	Filter doesn't match any resource	Verify filters, check resource exists
45	`Error: Multiple configurations for provider`	Duplicate provider blocks	Remove duplicate, use aliases for multi-region
46	Null values in `for_each` map	Map contains null values	Filter nulls: `{ for k, v in var.map : k => v if v != null }`
47	`terraform plan -out=plan.out` corrupted	Binary file handling issue	Regenerate plan file
48	Workspace-specific variable not applied	Wrong workspace selected	`terraform workspace show` to verify
49	`Error: error configuring S3 Backend`	AWS credentials for backend expired	Refresh credentials, re-init
50	Import then plan shows changes	Imported state doesn't match code exactly	Adjust code to match imported state, or accept the diff

Part 4 — Docker Deep Dive

Probability: 🔴 95% — Very Frequently Asked

Dockerfile — Line by Line

```
# === Stage 1: Build ===
FROM node:18-alpine AS builder        # 1. Base image (alpine = minimal)
# Q: "Why alpine?" → Smaller image (5MB vs 900MB), less attack surface
# TRAP: "But alpine uses musl libc, not glibc — some native modules may break"

WORKDIR /app                          # 2. Set working directory
# Creates dir if not exists. All subsequent commands run from here

COPY package*.json ./                 # 3. Copy dependency files FIRST
# Q: "Why copy package.json separately?" → Docker layer caching!
# If source code changes but deps don't, this layer is cached

RUN npm ci --only=production          # 4. Install dependencies
# Q: "npm ci vs npm install?" → ci uses lockfile exactly, faster, deterministic
# This layer is cached if package.json unchanged

COPY . .                              # 5. Copy source code
RUN npm run build                     # 6. Build application

# === Stage 2: Production ===
FROM node:18-alpine AS production     # 7. Fresh base — no build tools
# MULTI-STAGE: Only the final stage becomes the image

RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup    # 8. Non-root user
# SECURITY: Never run as root in production

WORKDIR /app

COPY --from=builder /app/dist ./dist           # 9. Copy ONLY build output
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./
# Result: No source code, no devDependencies, no build tools in final image

EXPOSE 3000                           # 10. Document the port
# Q: "Does EXPOSE open the port?" → NO. It's documentation only.
# You still need -p 3000:3000 at runtime

HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --spider http://localhost:3000/health || exit 1
# 11. Container health monitoring

USER appuser                          # 12. Switch to non-root
```

```
ENTRYPOINT ["node"]                    # 13. Fixed executable
CMD ["dist/server.js"]                # 14. Default arguments
```

ENTRYPOINT vs CMD — The Most Asked Docker Question

```
ENTRYPOINT = The executable (fixed)
CMD        = Default arguments (overridable)

Combined: ENTRYPOINT + CMD = full command

Example:
ENTRYPOINT ["node"]
CMD ["server.js"]
→ Runs: node server.js

Override CMD: docker run myimage worker.js
→ Runs: node worker.js

Override ENTRYPOINT: docker run --entrypoint python myimage script.py
→ Runs: python script.py
```

Instruction	Purpose	Overridable?
`ENTRYPOINT`	Defines the main executable	Only with `--entrypoint` flag
`CMD`	Default arguments to ENTRYPOINT	Yes, by appending args to `docker run`
Only `CMD`	Entire command + args	Yes, fully replaceable

Q: "When do you use ENTRYPOINT vs CMD?"

> *"Use ENTRYPOINT when the container is a specific tool (always runs node, or nginx). Use CMD for default behavior that users might override. For production microservices: ENTRYPOINT for the runtime + CMD for the script. This allows overriding the script for debugging without changing the runtime."*

Multi-Stage Build — Real Production Example

```
# Stage 1: Dependencies (cached heavily)
FROM golang:1.21-alpine AS deps
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download                    # Cached if go.mod unchanged

# Stage 2: Build
FROM deps AS builder
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /app/server ./cmd/server
#    ^^^^^^^^^^^^                ^^^^^^^^^^^^^^^^^
#    Static binary               Strip debug info (smaller binary)
```

```
# Stage 3: Production (FROM SCRATCH = empty image)
FROM scratch                          # 0 bytes base image!
COPY --from=builder /app/server /server
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
EXPOSE 8080
ENTRYPOINT ["/server"]
# Final image: ~10MB vs ~1GB with full golang base
```

Q: "What is a multi-stage build and why use it?"

> *"Multi-stage builds use multiple FROM statements. Each FROM starts a new stage. Only the final stage becomes the image. Benefits: 1) Build tools (compilers, dev deps) stay in build stage — not shipped to production. 2) Smaller images (we reduced from ~1GB to ~15MB). 3) Reduced attack surface — fewer packages = fewer CVEs. 4) Faster deployments — smaller images pull faster."*

Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
      target: production              # Build specific stage
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
    depends_on:
      postgres:
        condition: service_healthy    # Wait for health check
    networks:
      - app-network
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: '0.5'

  postgres:
    image: postgres:15-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data  # Named volume (persistent)
    environment:
      POSTGRES_DB: myapp
```

```
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password  # Docker secrets
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    networks:
      - app-network

volumes:
  pgdata:                              # Persists across container restarts

networks:
  app-network:
    driver: bridge
```

Docker Networking

```
NETWORK TYPES
┌─────────────────────────────────────────────┐
│ bridge (default)                            │
│ • Containers on same bridge can communicate │
│ • Isolated from host network                │
│ • Use for: single-host multi-container apps │
├─────────────────────────────────────────────┤
│ host                                        │
│ • Container shares host's network stack     │
│ • No port mapping needed                    │
│ • Use for: performance-critical apps        │
├─────────────────────────────────────────────┤
│ none                                        │
│ • No networking                             │
│ • Use for: batch jobs, security isolation   │
├─────────────────────────────────────────────┤
│ overlay                                     │
│ • Multi-host networking (Swarm/K8s)         │
│ • Use for: distributed applications         │
└─────────────────────────────────────────────┘
```

Q: "How do containers in the same Docker Compose communicate?"

> *"Docker Compose creates a default bridge network. Containers resolve each other by service name as hostname. So `app` can reach `postgres` at `postgres:5432`. No IP addresses needed. This is*

> *Docker's built-in DNS."*

Docker Volumes

Type	Syntax	Persistence	Use Case
Named Volume	`volumes: pgdata:/var/lib/data`	Persists after container removal	Database storage
Bind Mount	`volumes: ./src:/app/src`	Host directory	Development hot-reload
tmpfs	`tmpfs: /tmp`	Memory only, lost on stop	Sensitive temp data

Q: "Named volume vs bind mount?"

> *"Named volumes are managed by Docker — portable, backed up easily, work on all platforms. Bind mounts map a host directory directly — great for development but not portable. In production, always use named volumes. In development, bind mounts enable hot-reload."*

Image Optimization Best Practices

Practice	Impact	Example
Use alpine/slim bases	5MB vs 900MB	`FROM node:18-alpine`
Multi-stage builds	Remove build tools	See example above
Order layers by change frequency	Better caching	COPY deps → install → COPY source
Combine RUN commands	Fewer layers	`RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*`
Use .dockerignore	Smaller build context	Ignore `node_modules`, `.git`, `*.md`
Pin image versions	Reproducibility	`FROM node:18.17.1-alpine` not `FROM node:latest`
Don't install unnecessary packages	Smaller image, fewer CVEs	`--no-install-recommends`
Use `COPY` not `ADD`	Predictable behavior	`ADD` has extra features (URL fetch, tar extract) you rarely need

.dockerignore

```
node_modules
.git
.env
*.md
docker-compose*.yml
Dockerfile
.dockerignore
coverage/
tests/
```

Security Best Practices

1. **Never run as root** — `USER nonroot` or `USER 1000`
2. **Scan images** — `grype myimage:tag`, `trivy image myimage:tag`
3. **Pin base image digests** — `FROM node:18-alpine@sha256:abc123...`
4. **No secrets in images** — Use runtime env vars or mounted secrets
5. **Read-only filesystem** — `docker run --read-only`
6. **Drop capabilities** — `docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE`
7. **Use distroless/scratch** — Minimal attack surface for compiled languages

---

Top 30 Docker Interview Questions

#	Question	Key Points
1	Difference between image and container?	Image = blueprint (read-only). Container = running instance
2	Docker layers — how do they work?	Each instruction creates a layer. Layers are cached and shared
3	COPY vs ADD?	COPY is simple copy. ADD can fetch URLs and extract tars. Use COPY
4	ENTRYPOINT vs CMD?	See detailed section above
5	What is a dangling image?	Untagged image, usually old build. `docker image prune` to clean
6	How to reduce image size?	Multi-stage, alpine, .dockerignore, combine RUN
7	Docker networking types?	bridge, host, none, overlay
8	What is Docker Compose?	Multi-container orchestration tool using YAML
9	How do you persist data?	Named volumes for production, bind mounts for dev
10	What is a health check?	Container-level health monitoring. Restarts if unhealthy
11	Docker vs Podman?	Podman is daemonless, rootless by default, OCI-compatible
12	What is a Docker registry?	Image storage. DockerHub (public), ACR/ECR/GitLab (private)
13	Explain layer caching	Unchanged layers are cached. Order matters — volatile last

#	Question	Key Points
14	How do you handle secrets?	Runtime env vars, Docker secrets, mounted volumes. Never in image
15	What is build context?	Directory sent to Docker daemon for build. Minimize with .dockerignore
16	Explain `docker system prune`	Removes stopped containers, unused networks, dangling images, build cache
17	What is a scratch image?	Empty base image — 0 bytes. For statically compiled binaries
18	How does Docker DNS work?	Built-in DNS server resolves container names on user-defined networks
19	Docker Swarm vs Kubernetes?	Swarm is simpler but less feature-rich. K8s is industry standard
20	What is `docker exec`?	Run a command in a running container. Debug tool
21	How to debug a crashed container?	`docker logs`, `docker inspect`, `docker cp` to extract files
22	What are cgroups and namespaces?	cgroups = resource limits. namespaces = isolation (PID, network, mount)
23	Multi-architecture builds?	`docker buildx build --platform linux/amd64,linux/arm64`
24	Docker rate limiting?	DockerHub limits pulls. Solution: use private registry or mirror
25	Container vs VM?	Container: process isolation, shared kernel, lightweight. VM: full OS
26	What is `docker commit`?	Create image from running container. Anti-pattern — use Dockerfiles
27	How to limit container resources?	`--memory=512m --cpus=0.5` or resource limits in compose
28	What is OCI?	Open Container Initiative — standard for container images and runtimes
29	How to handle logging?	Log to stdout/stderr. Collect with Fluent Bit/Fluentd. Don't log to files
30	Dockerfile best practices for CI/CD?	Cache deps layer, multi-stage, tag with Git SHA, scan in pipeline

Part 5 — Kubernetes Deep Dive

Probability: 🔴 90% — Very Frequently Asked

Interview-Ready YAMLs

Pod (Foundation of Everything)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: default
  labels:                           # Labels = how K8s identifies pods
    app: nginx                      # Used by Services, Deployments
    version: "1.0"
spec:
  containers:
  - name: nginx
    image: nginx:1.25-alpine        # Always pin versions, never :latest
    ports:
    - containerPort: 80
    resources:                      # ALWAYS set in production
      requests:                     # Scheduling guarantee
        memory: "64Mi"
        cpu: "100m"                 # 100 millicores = 0.1 CPU
      limits:                       # Hard ceiling
        memory: "128Mi"
        cpu: "250m"
    livenessProbe:                  # Is the container alive?
      httpGet:
        path: /healthz
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 10
    readinessProbe:                 # Is it ready to serve traffic?
      httpGet:
        path: /ready
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
  restartPolicy: Always             # Always | OnFailure | Never
```

Q: "Should you create Pods directly?"

> *"Never in production. Always use Deployments (or StatefulSets/DaemonSets). Bare pods aren't rescheduled if a node fails. Deployments provide self-healing, rolling updates, and scaling."*

Deployment + ReplicaSet

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
```

```
  replicas: 3                       # Desired pod count
  selector:
    matchLabels:
      app: web-app                  # MUST match template labels
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0             # Zero-downtime: keep all old pods until new ones ready
      maxSurge: 1                   # Create 1 extra pod at a time
  template:                         # Pod template
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: myregistry/web-app:v2.1.0
        ports:
        - containerPort: 8080
        env:
        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: db_host
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: db_password
        resources:
          requests: { memory: "128Mi", cpu: "100m" }
          limits:   { memory: "256Mi", cpu: "500m" }
        readinessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 15
          periodSeconds: 20
```

Q: "What's the relationship between Deployment, ReplicaSet, and Pod?"

```
Deployment (manages)
  └── ReplicaSet (ensures N replicas running)
       ├── Pod 1
       ├── Pod 2
       └── Pod 3
```

> *"Deployment manages ReplicaSets. When you update image, Deployment creates NEW ReplicaSet, scales it up while scaling OLD one down. This enables rollback — old ReplicaSet is kept (scaled to 0). You never manage ReplicaSets directly."*

Services (ClusterIP, NodePort, LoadBalancer)

```
# ClusterIP — Internal only (default)
apiVersion: v1
kind: Service
metadata:
  name: web-app-svc
spec:
```

```
  type: ClusterIP                    # Only accessible within cluster
  selector:
    app: web-app                     # Matches pod labels
  ports:
  - port: 80                        # Service port
    targetPort: 8080                 # Container port
---
# NodePort — Expose on node IP
apiVersion: v1
kind: Service
metadata:
  name: web-app-nodeport
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080                  # External port (30000-32767)
---
# LoadBalancer — Cloud load balancer
apiVersion: v1
kind: Service
metadata:
  name: web-app-lb
spec:
  type: LoadBalancer                 # Creates cloud LB (ALB/NLB/Azure LB)
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 8080
```

Q: "ClusterIP vs NodePort vs LoadBalancer — when to use each?"

Type	Access	Use Case
ClusterIP	Internal only	Service-to-service communication
NodePort	`<NodeIP>:<30000-32767>`	Dev/testing, bare-metal
LoadBalancer	External cloud LB	Production external access
Ingress	HTTP/HTTPS routing	Production — one LB, multiple services

Ingress with TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls-cert         # TLS cert stored as K8s Secret
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
```

```
        pathType: Prefix
        backend:
          service:
            name: web-app-svc
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 8080
```

StatefulSet (for databases)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres"            # Headless service for stable DNS
  replicas: 3
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
      - name: postgres
        image: postgres:15-alpine
        ports: [{ containerPort: 5432 }]
        volumeMounts:
        - name: pgdata
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:              # Each pod gets its OWN PVC
  - metadata:
      name: pgdata
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: managed-premium
      resources:
        requests: { storage: 50Gi }
```

Q: "Deployment vs StatefulSet?"

Feature	Deployment	StatefulSet
Pod identity	Random names	Ordered: `postgres-0`, `postgres-1`
Storage	Shared or none	Each pod gets unique PVC
Startup order	Parallel	Sequential (0 → 1 → 2)
Stable DNS	No	Yes: `postgres-0.postgres.namespace.svc`
Use case	Stateless apps	Databases, Kafka, ZooKeeper

DaemonSet, Job, CronJob

```
# DaemonSet — One pod per node (monitoring agents, log collectors)
apiVersion: apps/v1
kind: DaemonSet
metadata:
```

```
  name: fluent-bit
spec:
  selector:
    matchLabels: { app: fluent-bit }
  template:
    metadata:
      labels: { app: fluent-bit }
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.1
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath: { path: /var/log }
---
# Job — Run to completion
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  backoffLimit: 3                    # Retry 3 times on failure
  template:
    spec:
      containers:
      - name: migrate
        image: myapp/migrate:v1
        command: ["python", "manage.py", "migrate"]
      restartPolicy: Never
---
# CronJob — Scheduled job
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "0 2 * * *"             # Daily at 2 AM
  concurrencyPolicy: Forbid          # Don't overlap runs
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: myapp/backup:v1
          restartPolicy: OnFailure
```

ConfigMap & Secret

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  db_host: "postgres-svc"
  log_level: "info"
  config.yaml: |                     # File-based config
    server:
      port: 8080
      timeout: 30s
---
apiVersion: v1
kind: Secret
```

```
metadata:
  name: app-secrets
type: Opaque
data:
  db_password: cGFzc3dvcmQxMjM=     # base64 encoded (NOT encrypted!)
  api_key: c2VjcmV0a2V5              # echo -n 'secretkey' | base64
```

Q: "Are Kubernetes Secrets secure?"

> *"By default, Secrets are base64 encoded — NOT encrypted. They're stored in etcd. To secure them: 1) Enable encryption at rest for etcd, 2) Use RBAC to restrict access, 3) Consider external secret managers (Vault, Azure Key Vault) with operators like External Secrets Operator. Base64 is encoding, not encryption — anyone with cluster access can decode them."*

HPA + ResourceQuota + LimitRange

```
# HPA — Scale pods based on metrics
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70       # Scale when avg CPU > 70%
---
# ResourceQuota — Limit namespace resources
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: team-a
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
---
# LimitRange — Default limits for pods
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-a
spec:
  limits:
  - default:                         # Default limit if not specified
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:                  # Default request if not specified
      cpu: "100m"
```

```
      memory: "128Mi"
    type: Container
```

NetworkPolicy + ServiceAccount

```
# NetworkPolicy — Restrict pod communication
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-network-policy
  namespace: production
spec:
  podSelector:
    matchLabels: { app: api }
  policyTypes: ["Ingress", "Egress"]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend }   # Only frontend can reach API
    ports:
    - port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels: { app: postgres }   # API can only reach postgres
    ports:
    - port: 5432
---
# ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
automountServiceAccountToken: false   # Don't auto-mount — security
```

InitContainer + Sidecar

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init-and-sidecar
spec:
  initContainers:                    # Runs BEFORE main container
  - name: wait-for-db
    image: busybox
    command: ['sh', '-c', 'until nc -z postgres-svc 5432; do sleep 2; done']
    # Waits until postgres is reachable before starting app

  containers:
  - name: app                        # Main container
    image: myapp:v1
    ports: [{ containerPort: 8080 }]

  - name: log-shipper                # Sidecar — runs alongside main
    image: fluent/fluent-bit:2.1
    volumeMounts:
    - name: app-logs
      mountPath: /var/log/app

  volumes:
  - name: app-logs
    emptyDir: {}                     # Shared between app and sidecar
```

Deployment Strategies

```
# Rolling Update (default — zero-downtime)
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0               # Keep all old pods until new ready
    maxSurge: 1                     # Add 1 new pod at a time

# Blue-Green (using service selector switch)
# Deploy v2 alongside v1, then switch service selector
# kubectl patch svc web-app -p '{"spec":{"selector":{"version":"v2"}}}'

# Canary (gradual rollout)
# Deploy v2 with fewer replicas, split traffic
# v1: 9 replicas, v2: 1 replica = 10% canary
# Monitor metrics, then scale v2 up, v1 down
```

Kubernetes Troubleshooting — 100 Scenarios

Pod Failures (1-25)

#	Symptom	Commands	Root Cause	Fix
1	CrashLoopBackOff	`kubectl logs <pod> --previous`	App crash, bad config, missing env var	Fix app code/config, check env vars
2	ImagePullBackOff	`kubectl describe pod`	Wrong image name, no registry auth	Fix image name, create imagePullSecret
3	OOMKilled (exit 137)	`kubectl describe pod` → last state	Memory limit too low	Increase memory limit or fix leak
4	Pending (no nodes)	`kubectl describe pod` → Events	Insufficient resources	Scale nodes or reduce requests
5	Pending (node affinity)	`kubectl describe pod`	No node matches nodeSelector/affinity	Fix labels or add matching nodes
6	Pending (taints)	`kubectl describe pod`	Node has taint, pod lacks toleration	Add toleration to pod spec
7	Pending (PVC)	`kubectl get pvc`	PVC not bound	Check StorageClass, provision PV
8	Init:Error	`kubectl logs <pod> -c <init-container>`	Init container failed	Fix init container command
9	CreateContainerConfigError	`kubectl describe pod`	Missing ConfigMap or Secret	Create the missing resource
10	RunContainerError	`kubectl describe pod`	Bad command, missing entrypoint	Fix container command
11	Evicted	`kubectl describe pod`	Node disk/memory pressure	Clean up disk, increase node resources
12	Error (exit code 1)	`kubectl logs <pod>`	Application error	Check application logs
13	Completed (not restarting)	Check restartPolicy	`restartPolicy: Never` set	Use `Always` for long-running services
14	Terminating (stuck)	`kubectl describe pod`	Finalizer stuck, or PV not releasing	`kubectl delete pod --force --grace-period=0`

#	Symptom	Commands	Root Cause	Fix
15	Unknown	`kubectl get nodes`	Node lost contact	Check kubelet on the node
16	Container keeps restarting	`kubectl logs --previous`	Liveness probe failing	Fix probe endpoint or increase thresholds
17	Pod slow to start	Check readiness probe timing	Large image pull or slow startup	Pre-pull images, use startup probe
18	Multi-container: one keeps crashing	`kubectl logs <pod> -c <container>`	Specific container issue	Debug that container only
19	Pod can't resolve DNS	`kubectl exec -- nslookup kubernetes`	CoreDNS issue	Check CoreDNS pods in kube-system
20	Pod can't reach external internet	`kubectl exec -- wget google.com`	NetworkPolicy blocking egress	Check/fix NetworkPolicy
21	Resource limits not applied	`kubectl describe pod`	LimitRange missing	Create LimitRange in namespace
22	Pod scheduled on wrong node	Check nodeSelector/affinity	Wrong labels	Fix labels/selectors
23	Env var is empty	`kubectl exec -- printenv`	ConfigMap/Secret key wrong	Verify key names in ConfigMap/Secret
24	Volume mount permission denied	Check securityContext	Container runs as wrong UID	Add `fsGroup` or `runAsUser`
25	Pod can't write to volume	Check PVC accessMode	ReadOnlyMany but needs ReadWrite	Use ReadWriteOnce or ReadWriteMany

Service & Networking (26-50)

#	Symptom	Commands	Root Cause	Fix
26	Service has no endpoints	`kubectl get endpoints <svc>`	Label selector mismatch	Fix service selector to match pod labels
27	Service not reachable	`kubectl get svc`, `kubectl get ep`	No running pods match selector	Fix pod labels or selector
28	Ingress returns 404	`kubectl describe ingress`	Path/host mismatch	Fix ingress rules
29	Ingress returns 502	Check backend pod health	Backend pod not ready	Fix readiness probe
30	Ingress TLS not working	Check TLS secret	Secret missing or wrong format	Create proper TLS secret
31	DNS resolution slow	Check CoreDNS metrics	CoreDNS overloaded	Scale CoreDNS, add node-local DNS
32	Pods can't communicate across namespaces	Check NetworkPolicy	NetworkPolicy blocking cross-namespace	Allow namespace in policy

#	Symptom	Commands	Root Cause	Fix
33	Service returns connection refused	Verify targetPort	targetPort doesn't match container port	Fix targetPort
34	LoadBalancer stuck at Pending	`kubectl describe svc`	Cloud provider issue	Check cloud quota/permissions
35	External traffic blocked	Check NSG/security groups	Cloud-level firewall	Allow port in cloud firewall
36	Headless service not resolving	Check StatefulSet	Pod DNS format: `pod-0.svc.ns.svc.cluster.local`	Use full DNS name
37	kube-proxy not routing	Check kube-proxy pods	kube-proxy crashed	Restart kube-proxy DaemonSet
38	CNI plugin failure	Check CNI pod logs	Flannel/Calico crash	Restart CNI pods, check node config
39	NodePort not accessible	Check node firewall	Firewall blocking port range	Open 30000-32767
40	ExternalName service not working	Check DNS resolution	Needs CoreDNS CNAME support	Verify ExternalName value
41	Session affinity not working	Check service config	Missing `sessionAffinity: ClientIP`	Add sessionAffinity
42	Slow service response	Check pod resources	CPU throttling	Increase CPU limits
43	Intermittent 503 errors	Check pod readiness	Pod cycling during updates	Fix rolling update strategy
44	gRPC through ingress fails	Check ingress annotations	Nginx needs gRPC annotation	Add `nginx.ingress.kubernetes.io/backend-protocol: GRPC`
45	WebSocket through ingress drops	Check timeout settings	Default timeout too low	Increase proxy-read-timeout
46	IP whitelisting not working	Check ingress annotations	Wrong annotation syntax	`nginx.ingress.kubernetes.io/whitelist-source-range`
47	Rate limiting not applied	Check ingress config	Missing annotation	Add rate-limit annotations
48	CORS issues	Check ingress CORS config	Missing CORS headers	Add CORS annotations to Ingress
49	SSL redirect loop	Check TLS termination	Both ingress and app doing TLS	Terminate at one layer only
50	Multiple ingress controllers conflict	Check ingressClassName	Both controllers claiming same ingress	Specify `ingressClassName` explicitly

Cluster & Node (51-75)

#	Symptom	Root Cause	Fix
51	Node NotReady	kubelet down	`systemctl restart kubelet`, check logs
52	Node disk pressure	Disk full	Clean images: `docker/crictl system prune`
53	Node memory pressure	RAM exhausted	Evict pods, scale node pool
54	Node PID pressure	Too many processes	Investigate runaway containers
55	etcd slow	Disk I/O bottleneck	Move etcd to SSD, monitor latency
56	API server unreachable	Certificate expired	Renew certs: `kubeadm certs renew all`
57	Scheduler not scheduling	Scheduler pod crashed	Check `/etc/kubernetes/manifests/kube-scheduler.yaml`
58	Controller manager down	Config error	Check static pod manifest
59	kubectl connection refused	kubeconfig wrong	Verify cluster endpoint and certs
60	RBAC permission denied	Missing RoleBinding	Create Role + RoleBinding
61	Can't create resources	ResourceQuota exceeded	Check quota: `kubectl get quota`
62	HPA not scaling up	Metrics server missing	Install metrics-server
63	HPA not scaling down	Cooldown period	Wait 5 min (default stabilization)
64	PVC stuck at Pending	No matching PV or StorageClass	Create PV or fix StorageClass
65	PV not releasing	`persistentVolumeReclaimPolicy: Retain`	Manually clear and release PV
66	Node cordoned accidentally	`kubectl cordon` was run	`kubectl uncordon <node>`
67	Pods not rescheduled after node drain	PDB blocking	Check PodDisruptionBudget
68	Admission webhook blocking	Webhook rejecting requests	Check webhook logs, bypass with dry-run
69	Namespace stuck at Terminating	Finalizer stuck	Remove finalizer from namespace
70	Secret not updating in pod	Pod doesn't re-read secrets	Restart pod or use volume-mounted secrets (auto-update)
71	ConfigMap change not reflected	Pod cached old value	Restart pod or use volume mount
72	Rolling update stuck	New pods failing readiness	Check new image, rollback: `kubectl rollout undo`
73	Helm release failed	Pre-install hook failed	`helm rollback`, fix hook
74	CRD not found	CRD not installed	`kubectl apply -f crds/`
75	Webhook timeout	Webhook service unreachable	Check webhook service and endpoints

Advanced (76-100)

#	Symptom	Root Cause	Fix
76	Pod security policy blocking	PSP/PSA restrictions	Update pod security context
77	Sidecar not terminating	Main container exits, sidecar runs	Use native sidecar containers (K8s 1.28+)
78	Jobs not completing	Container hangs	Add `activeDeadlineSeconds`
79	CronJob missed schedule	>100 missed schedules	Set `startingDeadlineSeconds`
80	Service mesh (Istio) breaking traffic	mTLS misconfigured	Check PeerAuthentication policy
81	Prometheus can't scrape pods	Missing annotations	Add `prometheus.io/scrape: "true"`
82	Grafana dashboard empty	Wrong data source	Verify Prometheus URL in datasource
83	Loki not receiving logs	Fluent Bit config wrong	Check Fluent Bit output config
84	Alertmanager not sending alerts	SMTP/Slack config wrong	Test with `amtool`
85	Horizontal scaling too aggressive	HPA threshold too low	Increase target utilization
86	Cluster autoscaler not adding nodes	Node pool max reached	Increase max nodes in pool
87	Image pull slow	Large images, no caching	Use smaller images, pre-pull
88	Pod affinity not respected	Soft affinity (preferred)	Use `requiredDuringScheduling` if strict
89	Topology spread not working	Labels mismatch	Check `topologySpreadConstraints`
90	Init container timeout	Dependency not available	Add timeout and retry logic
91	Container can't read mounted secret	File permissions	Set `defaultMode: 0400` in volume
92	Ephemeral storage exhausted	Too many logs in container	Set `ephemeral-storage` limits
93	kubectl slow	Large cluster, many objects	Use `--field-selector` to filter
94	Audit logs not working	API server audit config missing	Configure audit policy
95	Custom metrics HPA not working	Prometheus adapter not configured	Install prometheus-adapter
96	Blue-green rollback needed	New version has bug	Switch service selector back to old version
97	Canary showing errors	New version failing	Scale canary to 0, keep stable version
98	GitOps (ArgoCD) out of sync	Manual changes in cluster	Enable auto-sync or reconcile manually
99	Priority class evicting pods	Wrong PriorityClass assigned	Review and fix PriorityClass values
100	Cluster upgrade fails	Version skew too large	Upgrade incrementally (1.27→1.28→1.29)

Part 6 — Linux Deep Dive

Probability: 🔴 90% — Very Frequently Asked

100 Linux Interview Questions

File System & Navigation (1-15)

#	Question	Answer / Command
1	Find all log files modified in last 24 hours	`find /var/log -name "*.log" -mtime -1`
2	Find files larger than 100MB	`find / -type f -size +100M 2>/dev/null`
3	Find and delete empty directories	`find /path -type d -empty -delete`
4	Count files in a directory recursively	`find /path -type f | wc -l`
5	What's the difference between hard and soft links?	Hard link: same inode, can't cross filesystems. Soft link: pointer to path, can cross FS, can link dirs
6	Create a symbolic link	`ln -s /target/file /link/name`
7	What is an inode?	Metadata structure storing file info (permissions, owner, timestamps, block pointers). NOT the filename
8	Difference between `/tmp` and `/var/tmp`?	`/tmp` cleared on reboot. `/var/tmp` persists across reboots
9	What is `/proc`?	Virtual filesystem exposing kernel/process info. `/proc/cpuinfo`, `/proc/meminfo`
10	What is `/dev/null`?	Bit bucket — discards all data written to it. `command 2>/dev/null`
11	Explain the Linux directory hierarchy	`/` root, `/bin` binaries, `/etc` config, `/home` users, `/var` variable data, `/tmp` temp, `/opt` optional software
12	What does `df -h` show?	Disk filesystem usage (human-readable)
13	What does `du -sh /var/log` show?	Disk usage of specific directory
14	How to check which process has a file open?	`lsof /path/to/file`
15	What is `/etc/fstab`?	Filesystem mount table — defines auto-mounts at boot

Permissions (16-25)

#	Question	Answer
16	Explain `chmod 755`	Owner: rwx (7), Group: r-x (5), Others: r-x (5)
17	What is SUID?	SetUID — file runs as owner, not executor. `chmod u+s file`. Example: `/usr/bin/passwd`
18	What is SGID?	SetGID — file runs as group owner. On directory: new files inherit group
19	What is sticky bit?	On directory: only owner can delete files. Example: `/tmp` has sticky bit
20	Explain `umask 022`	Default permissions = 777 (dirs) or 666 (files) minus umask. 666-022 = 644 for files
21	`chown -R www-data:www-data /var/www`	Recursively change owner and group
22	What is ACL?	Access Control List — fine-grained permissions beyond owner/group/other. `setfacl`, `getfacl`
23	What does `chmod +x script.sh` do?	Adds execute permission for all (user, group, other)
24	Why can't a user delete a file in `/tmp` owned by another user?	Sticky bit is set on `/tmp`
25	How to find all SUID files?	`find / -perm -4000 -type f 2>/dev/null` — security audit command

Process Management (26-40)

#	Question	Answer / Command
26	List all processes	`ps aux` or `ps -ef`
27	Find process by name	`ps aux | grep nginx` or `pgrep -a nginx`
28	Kill a process gracefully	`kill <PID>` (sends SIGTERM — allows cleanup)
29	Force kill a process	`kill -9 <PID>` (SIGKILL — immediate, no cleanup)
30	What is a zombie process?	Process that has finished but parent hasn't collected exit status. Shows as `Z` in `ps`
31	How to fix zombie processes?	Kill the parent process. Init will adopt and reap orphan zombies
32	What is `nohup`?	Runs command immune to hangup signal. Persists after terminal close
33	Difference between `&` and `nohup`?	`&` backgrounds but dies with terminal. `nohup` survives terminal close
34	What does `nice` do?	Sets process priority. `-20` highest, `19` lowest. `nice -n 10 command`

#	Question	Answer / Command
35	What is `top` ? Real-time info shown?	Real-time process monitor: CPU%, MEM%, load average, uptime, process list
36	Explain load average (1, 5, 15 min)	Number of processes waiting for CPU. On 4-core: load 4.0 = 100% utilized
37	What is `htop` vs `top` ?	`htop` is interactive, colored, tree view, easier to use
38	How to run a process in background?	`command &` or `Ctrl+Z` then `bg`
39	What is `strace` ?	Traces system calls. Debug why a process fails: `strace -p <PID>`
40	What is `lsof` ?	List open files. Find what process uses a port: `lsof -i :8080`

Systemd & Services (41-50)

#	Question	Command
41	Start/stop/restart a service	`systemctl start/stop/restart nginx`
42	Enable service at boot	`systemctl enable nginx`
43	Check service status	`systemctl status nginx`
44	View service logs	`journalctl -u nginx -f` (follow)
45	View boot logs	`journalctl -b` (current boot)
46	List all running services	`systemctl list-units --type=service --state=running`
47	List failed services	`systemctl --failed`
48	Reload systemd after unit file change	`systemctl daemon-reload`
49	What is a systemd unit file?	Configuration defining how to manage a service. Located in `/etc/systemd/system/`
50	Create a custom systemd service	See unit file example below

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target
Wants=postgresql.service

[Service]
Type=simple
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
```

```
StandardOutput=journal
StandardError=journal
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Networking (51-65)

#	Question	Command / Answer
51	Check IP address	`ip addr show` or `hostname -I`
52	Check open ports	`ss -tulnp` or `netstat -tulnp`
53	Check if port is reachable	`nc -zv host 443` or `curl -v telnet://host:443`
54	Check DNS resolution	`nslookup domain.com` or `dig domain.com`
55	Check routing table	`ip route show` or `route -n`
56	Trace packet path	`traceroute host` or `mtr host`
57	Check network interfaces	`ip link show`
58	What is `iptables`?	Linux firewall. Rules organized in chains: INPUT, OUTPUT, FORWARD
59	Block an IP with iptables	`iptables -A INPUT -s 192.168.1.100 -j DROP`
60	What is `firewalld`?	Dynamic firewall manager (frontend to nftables). `firewall-cmd --add-port=80/tcp`
61	Check TCP connections	`ss -tn` — shows established connections
62	What is `/etc/hosts`?	Local DNS override. Maps hostnames to IPs
63	What is `/etc/resolv.conf`?	DNS resolver configuration. `nameserver 8.8.8.8`
64	What's the difference between TCP and UDP?	TCP: reliable, ordered, connection-oriented. UDP: fast, no guarantee, connectionless
65	What is a network namespace?	Isolated network stack — used by Docker/K8s containers

Disk & Storage (66-75)

#	Question	Command
66	Check disk space	`df -h`
67	Check directory size	`du -sh /var/log/*`
68	Find largest files	`du -ah / | sort -rh | head -20`
69	List block devices	`lsblk`
70	Mount a disk	`mount /dev/sdb1 /mnt/data`
71	Create a filesystem	`mkfs.ext4 /dev/sdb1`

#	Question	Command
72	Check disk I/O	`iostat -x 1`
73	What is LVM?	Logical Volume Manager — flexible disk management. Resize volumes without unmount
74	Extend an LVM volume	`lvextend -L +10G /dev/vg/lv && resize2fs /dev/vg/lv`
75	What is swap?	Disk space used as virtual memory when RAM is full. `swapon --show`

Text Processing (76-85)

#	Command	Usage
76	`grep -r "error" /var/log/`	Recursively search for "error" in logs
77	`grep -i -c "error" file.log`	Case-insensitive count of "error" lines
78	`grep -E "error|warning" file.log`	Extended regex — match error OR warning
79	`sed 's/old/new/g' file`	Replace all occurrences of "old" with "new"
80	`sed -i '/^#/d' config.conf`	Delete comment lines in-place
81	`awk '{print $1, $4}' access.log`	Print 1st and 4th columns
82	`awk '/ERROR/ {count++} END {print count}' app.log`	Count ERROR lines
83	`cut -d: -f1 /etc/passwd`	Extract usernames from passwd
84	`sort file | uniq -c | sort -rn`	Count unique lines, sort by frequency
85	`tail -f /var/log/syslog | grep error`	Follow logs, filter for errors

SSH & Security (86-92)

#	Question	Answer
86	Generate SSH key pair	`ssh-keygen -t ed25519 -C "email@example.com"`
87	Copy SSH key to server	`ssh-copy-id user@host`
88	SSH tunneling	`ssh -L 8080:localhost:3000 user@host` — access remote 3000 via local 8080
89	What is SSH agent forwarding?	Allows using your local SSH keys on remote servers. `ssh -A user@host`
90	Disable root SSH login	Edit `/etc/ssh/sshd_config`: `PermitRootLogin no`
91	Harden SSH	Disable password auth, use key-only, change port, use fail2ban

#	Question	Answer
92	What is fail2ban?	Monitors logs and bans IPs with too many failed login attempts

Cron & Scheduling (93-96)

#	Question	Answer
93	Edit crontab	`crontab -e`
94	Cron syntax	`MIN HOUR DOM MON DOW command` — `*/5 * * * *` = every 5 min
95	Run daily at 2 AM	`0 2 * * * /opt/scripts/backup.sh`
96	Check cron logs	`journalctl -u cron` or `grep CRON /var/log/syslog`

Real Troubleshooting Scenarios (97-100)

#	Scenario	Diagnostic Commands	Resolution
97	Server slow, high CPU	`top` → identify process → `strace -p PID`	Kill runaway process, optimize code
98	Disk full, can't even run commands	`du -sh /var/log/*`, `lsof | grep deleted`	Delete old logs, truncate large files: `> /var/log/huge.log`
99	Can't SSH into server	`ping host`, `nc -zv host 22`, `journalctl -u sshd`	Check firewall, sshd status, disk space
100	High memory, system swapping	`free -h`, `ps aux --sort=-%mem | head`	Identify memory hog, increase RAM or optimize

---

Quick Command Cheat Sheet

```
# Performance
top / htop                    # Real-time process monitor
vmstat 1                      # Virtual memory stats every 1s
iostat -x 1                   # Disk I/O stats
free -h                       # Memory usage
uptime                        # Load average

# Files
find / -name "*.log" -mtime -1    # Files modified in last day
locate filename                    # Fast file search (uses db)
tar -czf archive.tar.gz /path     # Create compressed archive
tar -xzf archive.tar.gz           # Extract archive

# Network
curl -v https://api.example.com   # HTTP request with verbose
wget -q -O- http://localhost/health  # Quick health check
ss -tulnp                         # Open ports with process names
```

```
ip addr                          # Network interfaces
dig +short example.com           # DNS lookup
```

Part 7 — Git, Monitoring & Security

Chapter 18: Git

Probability: 🔴 90%

Top Git Interview Questions

#	Question	Answer
1	`git merge` vs `git rebase`?	Merge creates a merge commit preserving history. Rebase replays commits on top of target — linear history. Use merge for shared branches, rebase for feature branches before merge
2	What is a fast-forward merge?	When target branch has no new commits since branch point. No merge commit needed. `git merge --ff`
3	How to resolve merge conflicts?	Edit conflicted files (remove markers), `git add`, `git commit`. Use `git mergetool` for GUI
4	What is `git stash`?	Temporarily saves uncommitted changes. `git stash` → do other work → `git stash pop`
5	What is `git cherry-pick`?	Apply a specific commit from another branch. `git cherry-pick <sha>`
6	How to undo last commit?	`git reset --soft HEAD~1` (keep changes staged). `git reset --hard HEAD~1` (discard changes)
7	What is `git reflog`?	Log of all HEAD movements. Recovery tool — find lost commits after reset
8	Explain `.gitignore`	File listing patterns to exclude from tracking. Prevents committing secrets, build artifacts, node_modules
9	What is a detached HEAD?	HEAD points to a commit, not a branch. Commits here are "orphaned" unless you create a branch
10	`git fetch` vs `git pull`?	Fetch: downloads remote changes but doesn't merge. Pull: fetch + merge (or rebase with `--rebase`)
11	What are Git hooks?	Scripts triggered by Git events. `pre-commit` (lint), `pre-push` (tests), `commit-msg` (format)
12	How to squash commits?	`git rebase -i HEAD~N` → change `pick` to `squash`. Clean commit history before merge
13	What is `git bisect`?	Binary search through commits to find which introduced a bug. `git bisect start`, `good`, `bad`
14	Branching strategy for your team?	Trunk-based: short-lived feature branches, merge to main via PR. Feature flags for incomplete work

#	Question	Answer
15	How to remove sensitive data from Git history?	`git filter-branch` or `BFG Repo-Cleaner`. Then force push. Rotate the exposed credential immediately

Branching Strategies

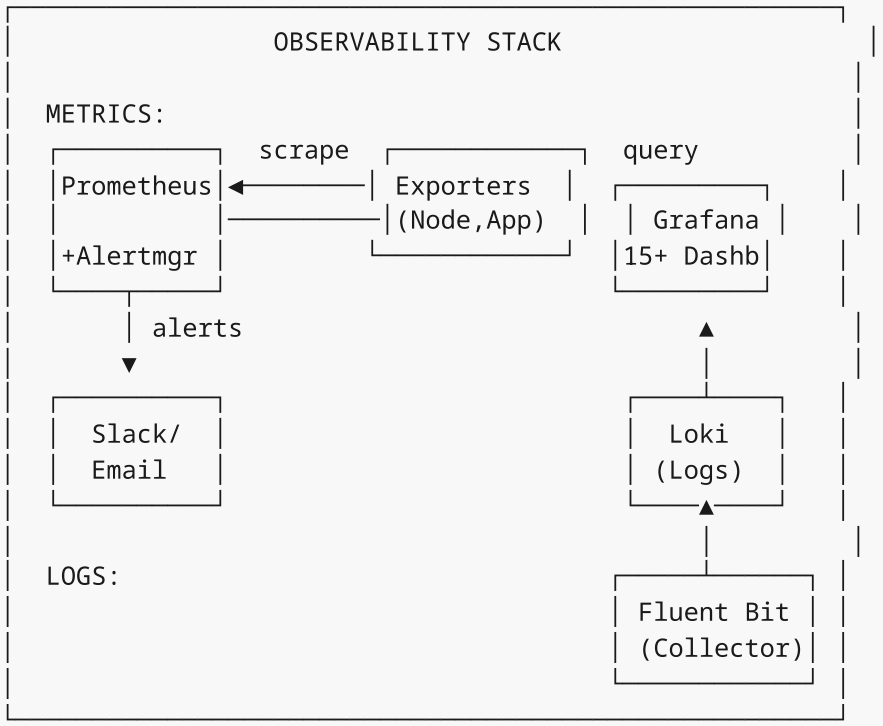
```
TRUNK-BASED (Recommended)
main ──●──●──●──●──●──●──●──●──●──
        \      /  \    /
feat-1   ●──●    feat-2 ●
(short-lived, 1-2 days)

GITFLOW (Complex teams)
main    ──●─────────────●─────●──
           \            / \     /
develop ──●──●──●──●──●   ●──●
             \  /
feature    ●──●
             release ──●──●
                       hotfix ──●
```

Chapter 19: Monitoring & Observability

Probability: 🟠 80%

Architecture (Your Setup)



Prometheus Concepts

Concept	Explanation
Pull-based	Prometheus SCRAPES targets (vs push-based like StatsD)
Metric types	Counter (always increases), Gauge (up/down), Histogram (distributions), Summary
PromQL	Query language. `rate(http_requests_total[5m])` = requests per second over 5 min
Service Discovery	Kubernetes SD, file-based, consul — automatically finds new targets
AlertManager	Receives alerts from Prometheus, deduplicates, groups, routes to Slack/email/PagerDuty
Recording Rules	Pre-compute expensive queries. Store results as new metrics

Key PromQL Examples

```
# Request rate per second
rate(http_requests_total[5m])

# 95th percentile latency
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Error rate percentage
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m])) * 100

# CPU usage per container
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)

# Memory usage percentage
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/ node_memory_MemTotal_bytes * 100

# Disk space alert condition
(node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 10
```

Interview Questions

Q: "How do you set up alerting for a production service?"

> *"I define SLO-based alerts: 1) Availability: alert if error rate > 1% for 5 minutes. 2) Latency: alert if p99 > 500ms for 10 minutes. 3) Saturation: alert if CPU > 80% or memory > 85%. These fire to Alertmanager which groups related alerts, deduplicates, and routes to Slack (warning) or PagerDuty (critical). I avoid alert fatigue by using multi-window burn rates rather than simple threshold alerts."*

Q: "Prometheus vs Azure Monitor — how do they compare?"

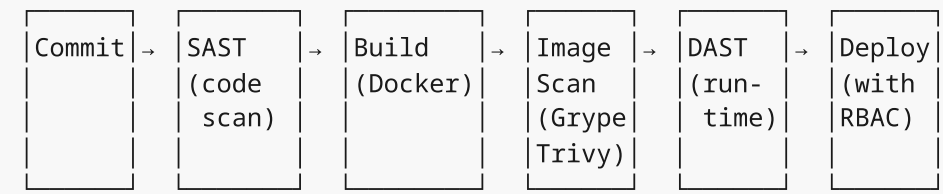
> *"Prometheus: open-source, pull-based, PromQL, runs in your cluster, you manage it. Azure Monitor: managed service, push-based (agents push to Azure), KQL query language, integrated with Azure*

> *services, auto-scales. My Prometheus+Grafana setup gives me the same capabilities. The advantage of managed monitoring (Azure Monitor) is zero operational overhead."*

Chapter 20: Security & DevSecOps

Probability: 🟡 70%

DevSecOps Pipeline



Key Security Topics

Topic	Your Experience	Interview Answer
Image Scanning	Grype in CI/CD pipeline	"We integrated Grype into our CI/CD security gate. Builds with critical/high CVEs are blocked. This reduced production vulnerability exposure by 90%."
RBAC	Kubernetes RBAC, Jenkins roles	"We use principle of least privilege. Kubernetes Roles scoped to namespaces, ClusterRoles for cluster-wide. Service accounts have minimal permissions."
Secrets	K8s Secrets, Jenkins credentials	"Never in code. K8s Secrets (encrypted at rest), Jenkins Credential Store, environment variables from CI/CD. For Azure: Key Vault with managed identity."
Network Security	NSGs, Security Groups, NetworkPolicy	"Defense in depth: cloud NSGs at network layer, K8s NetworkPolicies at pod layer, application-level auth."
Compliance	ISO 27001, SOC 2 support	"Supported ISO 27001 and SOC 2 compliance by enforcing access controls, centralized logging, backup validation, and change management workflows."
SSL/TLS	Certificate management	"TLS everywhere. cert-manager for automated Let's Encrypt certs in K8s. Internal mTLS for service-to-service."

Interview Questions

Q: "How do you handle a leaked secret in a Git repository?"

> *"1) Immediately rotate the credential. 2) Remove from Git history using BFG Repo-Cleaner. 3) Force push to all branches. 4) Audit access logs for the compromised credential. 5) Add pre-commit hooks to*

> *prevent future leaks (tools like git-secrets, detect-secrets). 6) RCA and team training. The credential rotation is the FIRST step — removing from Git history is important but the secret is already compromised."*

Q: "What is shift-left security?"

> *"Moving security earlier in the development lifecycle instead of only checking at deployment. Examples: SAST (static analysis) at commit time, dependency scanning in CI, container image scanning during build, policy-as-code for infrastructure. Our Grype integration is a shift-left example — we catch CVEs at build time, not in production."*

Q: "Explain RBAC in Kubernetes"

```
# Role — namespaced permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
# RoleBinding — binds Role to user/group/SA
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: production
  name: read-pods
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

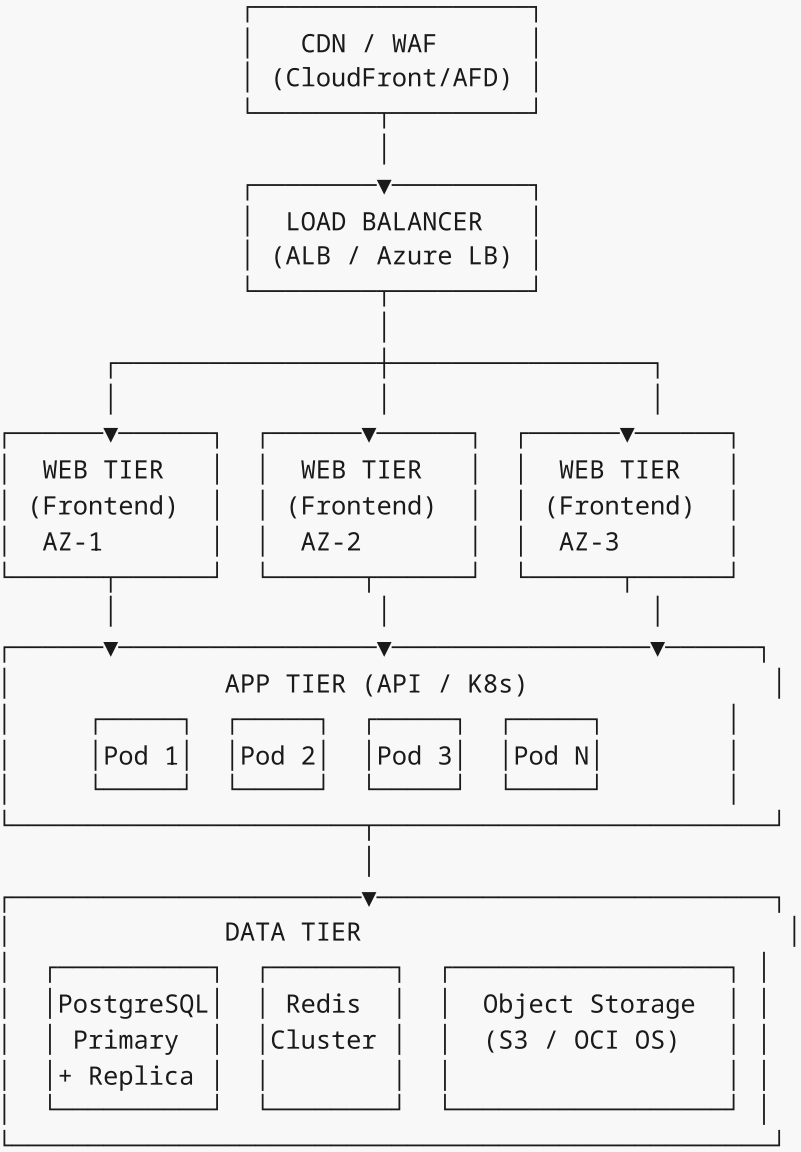
> *"Role defines WHAT permissions. RoleBinding defines WHO gets them. Role is namespace-scoped. ClusterRole is cluster-wide. Always use least privilege — give only the permissions needed."*

Part 8 — Architecture, HA, DR & Scalability

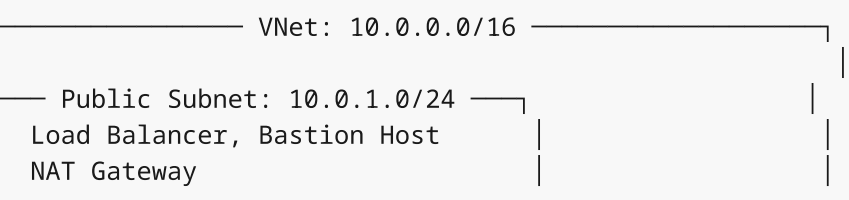
Chapter 21: Cloud Architecture

Probability: 🟡 65%

Multi-Tier Architecture



VPC/VNet Design



```
│   └──────────────────────────────────────┘                    │
│                                                               │
│   ┌─── Private Subnet (App): 10.0.10.0/24 ──┐                 │
│   │  Application Servers / K8s Nodes        │                 │
│   │  No public IPs — egress via NAT Gateway │                 │
│   └─────────────────────────────────────────┘                 │
│                                                               │
│   ┌─── Private Subnet (Data): 10.0.20.0/24 ──┐                │
│   │  Databases, Redis, Internal Services     │                │
│   │  No internet access                      │                │
│   └──────────────────────────────────────────┘                │
│                                                               │
│  NSG Rules:                                                   │
│  • Public: Allow 80/443 from internet                         │
│  • App: Allow from LB only                                    │
│  • Data: Allow from App subnet only                           │
└───────────────────────────────────────────────────────────────┘
```

Chapter 22: High Availability

Probability: 🟡 65%

HA Patterns

Pattern	Implementation	Your Experience
Multi-AZ	Deploy across availability zones	Kubernetes nodes across AZs
Database HA	Primary-replica with auto-failover	PostgreSQL + Patroni, Redis Sentinel
Load Balancing	Distribute traffic	Nginx, HAProxy, cloud LBs
Health Checks	Detect failures	K8s probes, LB health checks
Auto-healing	Replace failed instances	K8s restarts crashed pods
Stateless design	No server-side sessions	Sessions in Redis, state in DB

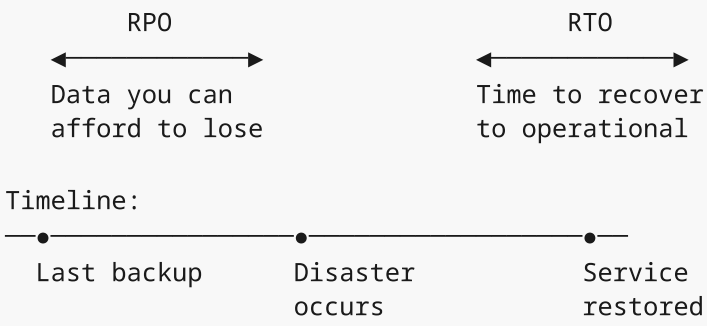
Q: "How do you achieve 99.9% uptime?"

> *"99.9% = 8.77 hours downtime per year. We achieve this through: 1) Multi-AZ deployment — no single point of failure. 2) Database HA with Patroni — automatic failover in seconds. 3) Kubernetes self-healing — crashed pods restart automatically. 4) Rolling updates with zero-downtime strategy. 5) Proactive monitoring with SLO-based alerts. 6) Documented runbooks for rapid incident response."*

Chapter 23: Disaster Recovery

Probability: 🟡 60%

RTO vs RPO



Tier	RPO	RTO	Strategy	Cost
Tier 1	~0	< 1 hour	Active-Active, synchronous replication	$$$$
Tier 2	< 1 hour	< 4 hours	Warm standby, async replication	$$$
Tier 3	< 24 hours	< 24 hours	Pilot light, scheduled backups	$$
Tier 4	> 24 hours	> 24 hours	Backup & restore only	$

Q: "Describe your DR strategy"

> *"We implement Tier 2 DR: automated snapshots every 6 hours with cross-region replication. PostgreSQL WAL archiving provides point-in-time recovery. Kubernetes manifests are in Git — we can recreate the entire application layer in a new region using Terraform + kubectl. Our tested RTO is sub-30 minutes. We run DR drills quarterly to validate."*

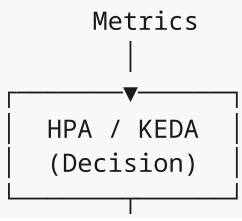
Chapter 24: Scalability

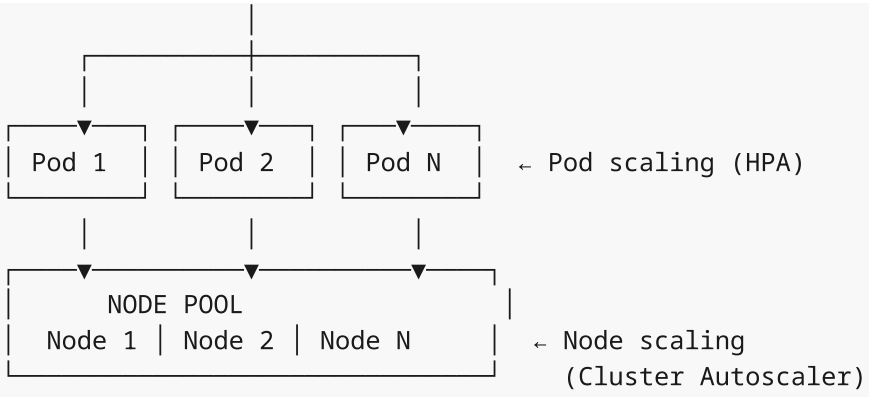
Probability: 🟡 55%

Horizontal vs Vertical Scaling

Type	How	Pros	Cons
Vertical	Bigger machine	Simple, no code changes	Single point of failure, hardware limits
Horizontal	More machines	Infinite scale, fault tolerant	Needs stateless design, complexity

Auto-Scaling Architecture





50 Architecture Interview Questions

#	Question	Key Points
1	Design a CI/CD pipeline for microservices	Monorepo vs polyrepo, parallel builds, artifact promotion, environment isolation
2	How would you migrate a monolith to microservices?	Strangler fig pattern, start with non-critical services, API gateway
3	Design a highly available web application	Multi-AZ, LB, auto-scaling, database replication, CDN
4	How do you handle database migrations in CI/CD?	Backward-compatible changes, separate migration step, rollback plan
5	Design a multi-environment infrastructure	Separate VNets, shared modules, environment-specific configs
6	How do you handle secrets across environments?	Key Vault per environment, separate service principals, RBAC
7	Design a monitoring strategy for 100+ microservices	Prometheus federation, Grafana dashboards per team, alert routing
8	How do you implement blue-green deployment?	Two identical environments, DNS/LB switch, instant rollback
9	How do you implement canary deployment?	Gradual traffic shift (1% → 5% → 25% → 100%), automated rollback on error rate
10	Design a disaster recovery plan	Multi-region, backup replication, automated failover, quarterly testing
11	How do you optimize cloud costs?	Rightsizing, reserved instances, spot/preemptible, idle cleanup, tagging
12	Design a logging architecture at scale	Fluent Bit → Loki/Elasticsearch, retention policies, structured logging
13	How do you handle certificate management?	cert-manager + Let's Encrypt for external, internal CA for mTLS

#	Question	Key Points
14	Design a GitOps workflow	Git as source of truth, ArgoCD/Flux, automated sync, drift detection
15	How do you secure a Kubernetes cluster?	RBAC, NetworkPolicy, PodSecurity, image scanning, audit logging
16	How do you handle noisy neighbor in multi-tenant K8s?	ResourceQuotas, LimitRanges, separate namespaces, priority classes
17	Design infrastructure for a startup going from 100 to 10,000 users	Start simple (managed services), plan for horizontal scaling, cache layer
18	How do you handle database schema versioning?	Flyway/Liquibase, backward compatible, CI-integrated
19	How do you implement zero-downtime deployments?	Rolling updates, readiness probes, connection draining, pre-stop hooks
20	Design a service mesh architecture	Istio/Linkerd for mTLS, traffic management, observability — but add complexity
21	How do you handle config across 50+ microservices?	Centralized config (ConfigMaps, Consul), environment-specific overrides
22	How do you test infrastructure changes safely?	Terraform plan review, staging environment, canary infrastructure
23	Design a backup strategy	3-2-1 rule (3 copies, 2 media, 1 offsite), automated, tested restores
24	How do you manage multiple Kubernetes clusters?	Fleet management (Rancher/ArgoCD), consistent policies, centralized monitoring
25	How do you implement RBAC at organization scale?	Groups/teams in AD/LDAP, mapped to K8s roles, audit regularly
26	Design an incident response system	PagerDuty/Opsgenie, runbooks, escalation matrix, blameless postmortems
27	How do you handle stateful applications in K8s?	StatefulSets, persistent volumes, operator pattern (e.g., PostgreSQL operator)
28	How do you implement IaC governance?	Sentinel/OPA policies, PR reviews, automated `tflint` / `checkov` in CI
29	Design a multi-cloud strategy	Abstract with Terraform modules, avoid cloud-specific lock-in where possible
30	How do you handle legacy application containerization?	Assess dependencies, create Dockerfile, test extensively, run in compatibility mode
31	Design API gateway architecture	Rate limiting, auth, routing, versioning, circuit breaker
32	How do you implement feature flags?	LaunchDarkly/Unleash, decouple deploy from release, gradual rollout

#	Question	Key Points
33	How do you handle data sovereignty/compliance?	Region-specific deployments, data residency controls, encryption, audit trails
34	Design a self-service platform for developers	Internal developer portal, golden path templates, automated environments
35	How do you measure DevOps maturity?	DORA metrics: lead time, deployment frequency, MTTR, change failure rate
36	How do you handle technical debt in infrastructure?	Allocate sprint capacity (20%), track in backlog, prioritize by risk
37	Design auto-scaling for unpredictable traffic	HPA + cluster autoscaler, pre-warming, KEDA for event-driven
38	How do you handle cross-region latency?	CDN, regional deployments, database read replicas per region
39	How do you secure CI/CD pipelines?	Signed commits, pipeline-as-code, secret masking, RBAC, audit logs
40	Design a platform engineering team's responsibilities	Golden paths, IDP, self-service infra, SRE practices, developer experience
41	How do you handle vendor lock-in?	Use open standards (K8s, Terraform), abstract cloud-specific APIs
42	Design observability for serverless	Distributed tracing, structured logging, custom metrics, X-Ray/Jaeger
43	How do you handle ephemeral environments?	Terraform workspaces or dynamic namespaces, auto-cleanup, PR-based
44	How do you implement chaos engineering?	Start with game days, Litmus/Chaos Monkey, controlled experiments, blast radius limits
45	How do you handle container image supply chain security?	Base image scanning, signed images, private registry, SBOM
46	Design a FinOps practice	Tagging strategy, cost dashboards, budget alerts, optimization reviews
47	How do you handle secrets rotation?	Automated rotation (Key Vault), short-lived tokens, workload identity
48	Design an SRE on-call rotation	Follow-the-sun, primary+secondary, runbooks, toil budget, escalation
49	How do you handle compliance audits?	Automated compliance checks, audit logs, policy-as-code, documentation
50	What's your approach to infrastructure documentation?	ADRs (Architecture Decision Records), runbooks, diagrams-as-code, living docs

Part 9 — Scenarios, Consulting, Agile, Manager & HR

Chapter 25: DevOps Consulting Questions

Probability: 🟡 60% (High for Infosys — they ARE a consulting company)

Q1: "How do you handle a client who wants to deploy directly to production without testing?"

> *"I'd first understand their urgency — is there a business-critical reason? Then I'd explain the risk quantitatively: 'In our experience, skipping testing leads to a 40% higher change failure rate.' I'd propose a compromise: an accelerated testing pipeline (automated tests only, skip manual QA) with an immediate rollback plan. Document the risk acceptance if the client still insists."*

Q2: "How do you onboard onto a client's existing infrastructure quickly?"

> *"1) Request architecture documentation and runbooks. 2) Get read-only access to monitoring dashboards. 3) Review the CI/CD pipelines and deployment process. 4) Shadow the existing team for 1-2 sprints. 5) Start with small improvements — build credibility before proposing major changes."*

Q3: "Client has 50+ microservices with no CI/CD — how do you approach this?"

> *"Don't try to fix everything at once. 1) Identify the 3-5 most frequently deployed services. 2) Build a golden pipeline template for one. 3) Prove value with metrics (deployment frequency, lead time). 4) Roll out template to remaining services. 5) Train the client's team for self-sufficiency."*

Q4: "How do you handle disagreements with a client's technical team?"

> *"Data over opinions. I present alternatives with trade-offs documented. If we still disagree, I escalate to get alignment at the architectural level. I document the decision (ADR — Architecture Decision Record) so the reasoning is preserved. Never make it personal."*

Chapter 26: Agile / Scrum

Probability: 🟡 60%

Concept	Definition	Interview Answer
Sprint	2-week iteration	"We run 2-week sprints with planning on Monday, demo on Friday"
Standup	Daily 15-min sync	"Three questions: what I did, what I'll do, any blockers"
Sprint Planning	Capacity + story selection	"We estimate in story points, plan based on team velocity"
Retrospective	Team improvement	"What went well, what didn't, action items for next sprint"
Backlog	Prioritized work list	"Product owner prioritizes, team estimates complexity"
Velocity	Points completed per sprint	"Our team averages 40-50 points per sprint"
DORA Metrics	DevOps performance	Lead Time, Deployment Frequency, MTTR, Change Failure Rate

---

Chapter 27: Scenario Questions (200+)

Production & Infrastructure (1-50)

#	Scenario	How to Answer
1	Production is down — what do you do first?	Check monitoring → identify affected service → check recent deployments → rollback if needed → communicate status
2	Database is unreachable	Check network → service status → disk space → connection limits → failover status
3	CPU at 100% on production server	`top` → identify process → check for runaway queries → resource limits → scale if needed
4	Disk full on production	`du -sh /*` → identify large files → rotate/delete logs → add disk → set up alerts
5	Memory leak in application	Monitor trend → identify growing process → heap dump → fix code → add memory limits
6	DNS resolution failing	Check `/etc/resolv.conf` → CoreDNS status → upstream DNS → network connectivity
7	SSL certificate expired	Immediate: replace cert → long-term: automate with cert-manager → monitor expiry
8	Load balancer health check failing	Check app health endpoint → verify port → check security groups → review LB config
9	Application responding slowly	Check latency metrics → identify bottleneck (CPU/memory/IO/network) → trace slow queries
10	Kubernetes node not ready	SSH → check kubelet → check disk/memory pressure → check certificates

#	Scenario	How to Answer
11	Pod in CrashLoopBackOff	`kubectl logs --previous` → check env vars → check mounts → check resource limits
12	Deployment rollback needed	`kubectl rollout undo deployment/app` → verify → RCA
13	Container image has critical vulnerability	Block deployment → patch base image → rebuild → rescan → deploy
14	Database replication lag increasing	Check network → check write volume → optimize queries → upgrade replica
15	Backup restore fails	Test with different backup → check corruption → verify backup process → fix and retest
16	Service mesh causing latency	Check sidecar resources → mTLS overhead → bypass for internal services if needed
17	Cloud provider outage	Activate DR plan → failover to secondary region → communicate to stakeholders
18	Pipeline running for 2+ hours	Check which stage is slow → cache dependencies → parallelize → optimize tests
19	Terraform state locked	Verify no one is running → `terraform force-unlock` → investigate cause
20	Secrets accidentally committed to Git	Rotate immediately → BFG cleaner → force push → add pre-commit hooks
21	High error rate after deployment	Rollback → check logs → compare config changes → fix → redeploy
22	Auto-scaler not scaling up	Check metrics server → HPA config → resource requests set → node pool limits
23	Network latency between services	Check network policies → CNI health → pod placement → consider pod affinity
24	Log volume overwhelming storage	Implement log levels → structured logging → retention policies → sampling
25	Monitoring gaps identified	Audit coverage → add missing exporters → create dashboards → set alerts

Security & Compliance (26-50)

#	Scenario	How to Answer
26	Security audit finds open ports	Audit all NSGs → close unnecessary ports → document exceptions → automate compliance checks
27	User reports unauthorized access	Check audit logs → identify scope → revoke access →

#	Scenario	How to Answer
		rotate credentials → investigate
28	Compliance requires audit trails for all changes	Implement GitOps → pipeline audit logs → cloud audit logging → centralized SIEM
29	Need to implement SOC 2 controls	Access controls (RBAC) → change management → logging → backup validation → encryption
30	Container running as root in production	Add securityContext → runAsNonRoot: true → PodSecurity admission → scan in CI
31-50	*(Cost optimization, budget constraints, legacy migration, client-specific technology constraints, team scaling, knowledge transfer, documentation gaps, tooling standardization, multi-cloud strategy, cross-team collaboration, vendor evaluation, capacity planning, performance testing, chaos engineering proposal, on-call rotation design, runbook creation, toil reduction, platform migration, technology refresh, compliance automation)*	Use STAR format with specific examples from your experience

Pipeline & Deployment (51-100)

#	Scenario	Key Points
51	Pipeline fails intermittently	Check agent resources → external dependency flakiness → network issues → add retries
52	Build times increased from 5 to 30 minutes	Profile stages → cache dependencies → parallelize → pre-bake agent images
53	Need to deploy to 4 environments	Parameterized pipeline → environment-specific variable groups → approval gates
54	Client wants daily releases but QA takes 3 days	Automate testing → shift-left → parallel test execution → risk-based testing
55	Merge conflicts blocking releases	Trunk-based development → smaller

#	Scenario	Key Points
		PRs → more frequent merges → feature flags
56-100	*(Database migration fails, rollback strategy, blue-green implementation, canary metrics, feature flag rollout, hotfix process, release train, multi-region deploy, config drift, infrastructure testing, pipeline security, artifact promotion, environment parity, data seeding, smoke tests, integration test isolation, deployment windows, change freeze, release notes automation, dependency updates, base image updates, helm chart versioning, GitOps sync issues, approval bottlenecks, pipeline cost optimization)*	Focus on: diagnosis process, communication, resolution, prevention

---

Chapter 28: Behavioral Questions — STAR Format (100)

Leadership & Teamwork (1-25)

#	Question
1	Tell me about a time you led a team through a difficult project
2	Describe a situation where you had to mentor a junior engineer
3	How did you handle a disagreement with a team member?
4	Tell me about a time you had to make a technical decision with incomplete information
5	Describe when you had to push back on a manager's request
6	How did you handle multiple priorities with tight deadlines?
7	Tell me about a time you failed and what you learned
8	Describe a time you improved a process that was inefficient
9	How did you onboard a new team member?
10	Tell me about driving adoption of a new tool/practice
11-25	*(Handling underperforming team member, cross-team collaboration, knowledge sharing, presenting to non-technical stakeholders, handling scope creep, working under pressure, receiving critical feedback, delegating effectively, managing remote team, building team culture)*

Problem Solving (26-50)

#	Question
26	Describe the most complex technical problem you've solved
27	Tell me about a production outage you resolved
28	How did you optimize a system's performance?
29	Describe a time you had to learn a new technology quickly

#	Question
30	Tell me about a time you identified a security vulnerability
31-50	*(Root cause analysis example, cost optimization win, automation initiative, migration project, debugging a hard-to-reproduce issue, handling legacy system, capacity planning, disaster recovery test, compliance implementation, monitoring gap discovery)*

STAR Template for YOUR Experience

```
SITUATION: "At Kochar Innovations, we were managing 15+ production
           applications with increasing deployment complexity..."

TASK:      "As the technical lead, I needed to standardize our
           deployment patterns across all teams..."

ACTION:    "I defined infrastructure standards and deployment patterns,
           created operational runbooks, and established platform best
           practices. I implemented these through PR reviews,
           documentation, and team training sessions..."

RESULT:    "Deployment consistency improved significantly — we reduced
           deployment failures by 40% and cut mean time to deploy
           from 45 minutes to 12 minutes. These practices were
           adopted across all application teams."
```

Chapter 29: Manager Round

Probability: 100% at Infosys

Key areas tested: Leadership, conflict resolution, client handling, Agile, career goals

Question	How to Answer
"Why Infosys?"	"Enterprise scale, diverse client projects, Azure/cloud practice growth, professional development"
"How do you handle conflicts in a team?"	STAR: describe specific conflict → your mediation approach → outcome
"How do you ensure quality with tight deadlines?"	"Automate testing, prioritize critical path, communicate risks early, never cut on security"
"How do you handle a client escalation?"	"Acknowledge → investigate → communicate timeline → resolve → prevent recurrence"
"Where do you see yourself in 5 years?"	"Technical architect / principal DevOps engineer, leading platform engineering initiatives at enterprise scale"
"How do you stay updated with technology?"	"Tech blogs, conferences, personal projects (GitHub), certifications, community participation"

Chapter 30: HR Round (50 Questions)

#	Question	Strategy
1	Tell me about yourself	Professional version — 60 seconds max
2	Why are you leaving your current role?	Growth opportunity, enterprise scale — NEVER badmouth current employer
3	Why Infosys?	Research their DevOps practice, recent wins, Azure partnership
4	What are your salary expectations?	"Based on market research for this role and my experience, I'm looking at X-Y LPA"
5	What is your notice period?	State clearly. Mention if negotiable
6	Are you open to relocation?	Yes — your resume says "Open to Relocation"
7	Do you have other offers?	If yes, mention professionally as it shows market demand
8	What's your expected CTC?	Research: Infosys Sr DevOps 4YOE = 18-28 LPA range
9	How do you handle work pressure?	Prioritize, communicate, stay organized, take breaks
10	Any questions for us?	Always have 2-3: team structure, tech stack, growth path
11-50	*(Strengths/weaknesses, work-life balance, travel willingness, team size preference, remote work, shift flexibility, long-term commitment, leadership style, learning from failures, handling ambiguity, diversity, company culture fit)*	

Chapter 31: Salary Discussion

Market Research for Sr DevOps Engineer at Infosys (India, 2025-2026)

Experience	Base Range (LPA)	Notes
3-5 YOE	15-22 LPA	Your range
5-8 YOE	20-30 LPA	With strong K8s/Azure

Experience	Base Range (LPA)	Notes
8+ YOE	28-40+ LPA	Lead/Architect

Negotiation Strategy

1. **Know your number** — Research on levels.fyi, Glassdoor, 6figr.com
2. **State a range, not a single number** — "I'm looking at 20-24 LPA based on my experience and market benchmarks"
3. **Focus on total CTC** — Base + Variable + Joining bonus + Benefits
4. **Leverage competing offers** — "I have an offer at X for Y LPA, but I prefer Infosys because..."
5. **Don't accept the first offer** — Infosys has approval layers, the first number is rarely the best
6. **Ask about growth** — "What does the salary progression look like in 12-18 months?"

What to Say

> *"Based on my 4+ years of experience as a Technical Lead managing 8+ engineers, my expertise in Kubernetes, Terraform, and production infrastructure, and current market rates for this role, I'm targeting 20-24 LPA. I'm open to discussing the overall compensation structure including variable pay and benefits."*

What NOT to Say

- "I'll take whatever you offer" → You'll get the minimum
- "My current CTC is X" → They'll anchor to your current, not your market value
- "I need at least X because of my expenses" → Personal needs ≠ market value

Part 10 — Cheat Sheets, Revision & Mock Interviews

One-Page Cheat Sheets

1. Azure DevOps Cheat Sheet

```
AZURE DEVOPS = Repos + Pipelines + Boards + Artifacts + Test Plans
Pipeline Structure: trigger → pool → variables → stages → jobs → steps
Agents: Microsoft-hosted (managed, fresh VM) | Self-hosted (your infra, private acc
YAML Keywords: trigger, pool, variables, stages, jobs, steps, template, extends, pa
Service Connection: Auth to external services (Azure, K8s, Docker). Use Workload Id
Variable Groups: Env-specific config. Link to Key Vault for secrets
Environments: Deployment targets with approval gates, checks, and history
Templates: Reusable YAML. extends (enforce), template (include)
Classic vs YAML: Always YAML for new projects (version-controlled, reviewable)
Approvals: Configured on Environments, not pipelines
Key command: az pipelines run --name "MyPipeline" --branch main
```

2. Terraform Cheat Sheet

```
COMMANDS: init → plan → apply → destroy | fmt | validate | import | state
STATE: Remote backend (S3/Azure Blob) + locking (DynamoDB/blob lease)
VARIABLES: string, number, bool, list, map, object | sensitive = true
MODULES: Reusable code. Pin with ?ref=v1.0.0. Test with Terratest
COUNT vs FOR_EACH: for_each for distinct resources (no index shift problem)
LIFECYCLE: create_before_destroy, prevent_destroy, ignore_changes
DATA SOURCES: Read existing resources. data "azurerm_subnet" "existing" {}
WORKSPACES: Separate state per workspace. Better: separate directories
STATE MGMT: state list, state show, state mv, state rm, state pull
IMPORT: terraform import azurerm_resource_group.main /subscriptions/.../rg
DRIFT: terraform plan detects. Apply to reconcile or update code
SENSITIVE: Mark variables/outputs sensitive. Never in .tfvars in Git
BACKEND: azurerm { storage_account, container, key } | s3 { bucket, key, dynamodb }
```

3. Kubernetes Cheat Sheet

```
ARCHITECTURE: API Server → etcd, Scheduler, Controller Manager | kubelet, kube-prox
WORKLOADS: Pod < ReplicaSet < Deployment | StatefulSet | DaemonSet | Job | CronJob
SERVICES: ClusterIP (internal) | NodePort (30000-32767) | LoadBalancer | Ingress
STORAGE: PV → PVC → Pod. StorageClass for dynamic provisioning
CONFIG: ConfigMap (non-sensitive) | Secret (base64, NOT encrypted by default)
SCALING: HPA (pods) + Cluster Autoscaler (nodes). Requires metrics-server
PROBES: liveness (restart if fails) | readiness (remove from service) | startup (sl
STRATEGY: RollingUpdate (default) | Recreate. maxUnavailable + maxSurge
SECURITY: RBAC (Role/ClusterRole + Binding) | NetworkPolicy | PodSecurity | Service
DEBUGGING: kubectl describe → kubectl logs → kubectl exec → kubectl get events
```

```
ROLLBACK: kubectl rollout undo deployment/<name>
NAMESPACES: Isolation boundary. ResourceQuota + LimitRange per namespace
KEY COMMANDS:
  kubectl get pods -A                    # All namespaces
  kubectl describe pod <name>            # Detailed info
  kubectl logs <pod> -c <container> -f   # Follow logs
  kubectl exec -it <pod> -- /bin/sh      # Shell into pod
  kubectl top pods                       # Resource usage
  kubectl get events --sort-by='.lastTimestamp'  # Recent events
```

4. Linux Cheat Sheet

```
FILES: find, ls, cat, head, tail, wc, du, df, ln, chmod, chown
PROCESS: ps aux, top, htop, kill, kill -9, nice, nohup, strace, lsof
NETWORK: ip addr, ss -tulnp, netstat, curl, wget, dig, nslookup, traceroute, nc
SYSTEMD: systemctl start/stop/enable/status, journalctl -u <service> -f
DISK: df -h, du -sh, lsblk, mount, fdisk, mkfs, lvextend
TEXT: grep -rni, sed 's/old/new/g', awk '{print $1}', cut -d: -f1, sort | uniq -c
SSH: ssh-keygen, ssh-copy-id, ssh -L (tunnel), scp, rsync
CRON: crontab -e, format: MIN HOUR DOM MON DOW cmd
PERMISSIONS: chmod 755, chown user:group, umask 022, SUID/SGID/Sticky
LOGS: /var/log/syslog, /var/log/auth.log, journalctl, dmesg
TROUBLESHOOTING:
  High CPU:    top → identify PID → strace -p PID
  Disk Full:   du -sh /* → find large files → rotate logs
  Can't SSH:   ping → nc -zv host 22 → check sshd → check firewall
  OOM:         free -h → ps aux --sort=-%mem → kill/optimize
```

5. Docker Cheat Sheet

```
BUILD: docker build -t name:tag . | Multi-stage: FROM ... AS builder
RUN:   docker run -d -p 8080:80 --name app image:tag
EXEC:  docker exec -it container /bin/sh
LOGS:  docker logs -f container
COMPOSE: docker compose up -d | down | logs -f | ps
DOCKERFILE: FROM → WORKDIR → COPY deps → RUN install → COPY . → EXPOSE → CMD
ENTRYPOINT vs CMD: ENTRYPOINT=executable(fixed), CMD=args(overridable)
VOLUMES: Named(-v pgdata:/data) | Bind(-v ./src:/app) | tmpfs
NETWORK: bridge(default) | host | none | overlay(multi-host)
OPTIMIZE: alpine base, multi-stage, .dockerignore, combine RUN, pin versions
SECURITY: non-root USER, scan with grype/trivy, no secrets in image
CLEANUP: docker system prune -a --volumes
```

6. Git Cheat Sheet

```
BASICS: clone, add, commit, push, pull, fetch, status, log, diff
BRANCH: branch, checkout -b, switch, merge, rebase, cherry-pick
UNDO: reset --soft/--hard HEAD~1, revert (new commit), reflog (recover)
STASH: stash, stash pop, stash list, stash drop
STRATEGY: Trunk-based (recommended) | GitFlow (complex teams)
MERGE vs REBASE: merge=preserve history, rebase=linear history
CONFLICT: Edit files → remove markers → git add → git commit
HOOKS: pre-commit(lint), pre-push(test), commit-msg(format)
```

```
SQUASH: rebase -i HEAD~N → squash
BISECT: bisect start → good → bad (binary search for bug)
```

7-11: Azure, Monitoring, Security, Networking, CI/CD

```
AZURE: Resource Groups → VNet/Subnet/NSG → VMs/AKS/App Service
       Key Vault (secrets) | ACR (images) | Azure Monitor (metrics)
       Managed Identity > Service Principal > Access Keys

MONITORING: Prometheus(metrics) + Grafana(dashboards) + Loki(logs)
            AlertManager → Slack/PagerDuty. PromQL for queries
            SLI(metric) → SLO(target) → SLA(contract)
            rate(), histogram_quantile(), sum by()

SECURITY: Shift-left: SAST→Build→Scan→DAST→Deploy
          RBAC everywhere (K8s, Azure, CI/CD)
          Secrets: Key Vault > K8s Secrets > Env vars. Never in code
          Network: Defense in depth (NSG + NetworkPolicy + App auth)

NETWORKING: OSI model, TCP vs UDP, DNS, HTTP/HTTPS
            VNet peering, VPN Gateway, Private endpoints
            Load Balancer (L4) vs Application Gateway (L7)
            K8s: ClusterIP→NodePort→LoadBalancer→Ingress

CI/CD: Commit→Build→Test→Scan→Stage→Prod
       Pipeline-as-code (Jenkinsfile/YAML). Artifact immutability
       Environment promotion with approvals. Rollback strategy
       DORA metrics: Lead Time, Frequency, MTTR, Change Failure Rate
```

Last Night Revision

30-Minute Revision (Emergency)

1. **Tell me about yourself** — rehearse 90-second version (2 min)
2. **Project architecture** — draw diagram from memory (3 min)
3. **Top 5 Docker questions** — ENTRYPOINT vs CMD, multi-stage, layers (3 min)
4. **Top 5 K8s questions** — Deployment vs StatefulSet, Services, CrashLoopBackOff debug (3 min)
5. **Top 5 Terraform questions** — state locking, count vs for_each, modules (3 min)
6. **Azure DevOps mapping** — Repos=GitLab, Pipelines=Jenkins, Artifacts=Nexus (3 min)
7. **Linux troubleshooting** — high CPU, disk full, can't SSH (3 min)
8. **Incident handling** — STAR example ready (3 min)
9. **Why Infosys + salary expectation** — rehearse (2 min)
10. **Review cheat sheets above** (5 min)

1-Hour Revision

- Everything in 30-min + add:
- CI/CD pipeline design — draw end-to-end flow (5 min)

- Git branching strategy + merge vs rebase (5 min)
- Monitoring stack — Prometheus, PromQL examples (5 min)
- Security — DevSecOps pipeline, RBAC, secrets (5 min)
- HA/DR — 99.9% SLA strategy, RTO/RPO (5 min)
- 3 STAR stories ready (conflict, failure, achievement) (5 min)

3-Hour Revision

- Everything in 1-hour + add:
- Read through Part 2 (Azure DevOps) completely (20 min)
- Read through Part 3 (Terraform) — focus on advanced + troubleshooting (20 min)
- Read through Part 5 (Kubernetes) — YAMLs + troubleshooting top 25 (20 min)
- Practice 20 scenario questions aloud (20 min)
- Review architecture questions — practice explaining trade-offs (20 min)
- Review Docker Compose, volumes, networking (10 min)
- Practice "What are your weaknesses?" and other HR questions (10 min)
- Review Azure Key Vault, ACR, AKS comparisons (10 min)
- Rest and hydrate (10 min)

6-Hour Revision (Full Preparation)

- Hour 1: Behavioral + Project walkthrough practice (speak aloud)
- Hour 2: Azure DevOps (Part 2) — full read, practice YAML from memory
- Hour 3: Terraform (Part 3) — full read, practice resource blocks from memory
- Hour 4: Kubernetes (Part 5) — YAMLs + troubleshooting scenarios
- Hour 5: Linux (Part 6) + Docker (Part 4) — command practice
- Hour 6: Architecture (Part 8) + Scenarios (Part 9) + HR + salary prep
- Last 15 min: Cheat sheets review + deep breaths

Mock Interview Questions

Easy Level (50 Questions)

#	Question
1	What is DevOps?
2	What is CI/CD?
3	What is Docker?
4	Difference between container and VM?
5	What is Kubernetes?
6	What is a Dockerfile?

#	Question
7	What is `git merge`?
8	What is Terraform?
9	What is Infrastructure as Code?
10	What is a Linux process?
11	What does `chmod 755` mean?
12	What is a load balancer?
13	What is DNS?
14	What is SSH?
15	What is YAML?
16	What is a microservice?
17	What is monitoring?
18	What is Prometheus?
19	What is Grafana?
20	What is an environment variable?
21	What is a Git branch?
22	What is a pull request?
23	What is a deployment pipeline?
24	What is a reverse proxy?
25	What is Nginx?
26	What is a cron job?
27	What is systemd?
28	What is TCP vs UDP?
29	What is a VPN?
30	What is RBAC?
31	What is a secret in Kubernetes?
32	What is a ConfigMap?
33	What is `docker compose up`?
34	What is a volume in Docker?
35	What is `terraform init`?
36	What is `terraform plan`?
37	What is a resource in Terraform?

#	Question
38	What is a Kubernetes Pod?
39	What is a Kubernetes Service?
40	What is a namespace?
41	What is `kubectl`?
42	What is a Jenkins pipeline?
43	What is Ansible?
44	What is a firewall?
45	What is HTTPS?
46	What is base64 encoding?
47	What is a health check?
48	What is high availability?
49	What is a backup?
50	What is an SLA?

Medium Level (50 Questions)

#	Question
1	ENTRYPOINT vs CMD in Docker?
2	How does Docker layer caching work?
3	Deployment vs StatefulSet in K8s?
4	How do you debug CrashLoopBackOff?
5	`git merge` vs `git rebase`?
6	Explain Terraform state and why it matters
7	What is a multi-stage Docker build?
8	How does Kubernetes service discovery work?
9	Explain Prometheus pull-based model
10	What is a Kubernetes Ingress?
11	How do you handle secrets in CI/CD?
12	What is a rolling update strategy?
13	Explain count vs for_each in Terraform
14	What are Kubernetes probes and their types?
15	How do you optimize Docker image size?

#	Question
16	What is `terraform import`?
17	How do you handle merge conflicts?
18	What is an HPA in Kubernetes?
19	Explain PromQL `rate()` function
20	What is a NetworkPolicy?
21	How does Jenkins credential store work?
22	What is a Kubernetes DaemonSet?
23	Explain Azure DevOps Service Connections
24	What is `terraform workspace`?
25	How do you implement blue-green deployment?
26	What is Docker Compose networking?
27	Explain Kubernetes RBAC
28	What is a Terraform module?
29	How do you set up alerting with Alertmanager?
30	What is PV and PVC in Kubernetes?
31	How do you handle log aggregation at scale?
32	What is a Kubernetes Job vs CronJob?
33	Explain branch protection rules
34	What is Azure Pipelines YAML vs Classic?
35	How do you handle database HA?
36	What is init container in K8s?
37	How do you secure SSH?
38	What is `terraform state rm`?
39	How does Kubernetes DNS work?
40	What is container vulnerability scanning?
41	Explain SLI vs SLO vs SLA
42	What is a sidecar pattern?
43	How do you manage Terraform state in a team?
44	What is Fluent Bit vs Fluentd?
45	How do you handle Kubernetes upgrades?
46	What is a Helm chart?

#	Question
47	Explain Docker networking modes
48	What is `terraform lifecycle`?
49	How do you implement canary deployments?
50	What is Kubernetes admission controller?

Hard Level (50 Questions)

#	Question
1	Design a CI/CD pipeline for a microservices architecture
2	Your Terraform state is corrupted — walk me through recovery
3	A pod keeps getting OOMKilled despite having 2GB limit — diagnose
4	Design a monitoring strategy for 100+ microservices
5	How would you implement zero-downtime database migration?
6	Explain how Patroni handles PostgreSQL failover
7	Design a multi-environment infrastructure with Terraform
8	Your Kubernetes cluster has DNS resolution failures — troubleshoot
9	How would you implement GitOps for 20+ applications?
10	Explain Terraform state locking internals
11	Design a disaster recovery plan with 30-minute RTO
12	Your Jenkins master is down — what's your recovery plan?
13	How do you handle secret rotation across 50+ services?
14	Design an auto-scaling strategy for unpredictable traffic
15	Your Docker build is 30 minutes — optimize it to 5 minutes
16	A Kubernetes node has disk pressure — walk through resolution
17	How do you implement compliance-as-code?
18	Design a logging pipeline that handles 1TB/day
19	Your deployment caused a 10% error rate increase — handle it
20	Explain AKS vs EKS vs self-managed Kubernetes trade-offs
21-50	*(Advanced networking, service mesh decisions, multi-cloud terraform, etcd backup/restore, K8s certificate renewal, custom Prometheus exporters, Grafana dashboard design, pipeline security hardening, cost optimization at scale, capacity planning methodology, chaos engineering implementation, platform engineering strategy, migration from monolith to microservices, container runtime comparison, Kubernetes operator pattern, advanced PromQL, Terraform provider development, incident response automation, SRE practices implementation, production readiness review)*

Expert Level (50 Questions)

#	Question
1	Design the entire infrastructure for a 500-developer organization
2	How would you build an Internal Developer Platform?
3	Architect a multi-region active-active deployment
4	Design a platform engineering team's operating model
5	How do you measure and improve DORA metrics at scale?
6	Design a Terraform module strategy for 100+ teams
7	Architect a zero-trust network for Kubernetes
8	How would you migrate 200 VMs to Kubernetes?
9	Design an observability platform supporting 10,000 containers
10	How do you implement progressive delivery at enterprise scale?
11-50	*(Enterprise RBAC strategy, multi-cluster management, FinOps at scale, compliance automation framework, developer experience measurement, SRE toil budget management, incident management maturity model, technical debt quantification, vendor evaluation framework, capacity planning for 10x growth, security audit automation, cloud cost governance, infrastructure testing strategy, change management at scale, release engineering practices, production readiness criteria, blameless postmortem culture, on-call health metrics, knowledge management system, technology radar process)*

Principal DevOps Level (50 Questions)

#	Question
1	How would you establish a DevOps Center of Excellence?
2	Design an organization-wide platform strategy
3	How do you drive cultural change for DevOps adoption?
4	Architect a global deployment pipeline for 30+ teams across 5 regions
5	How do you balance innovation with stability?
6	Design a technology governance framework
7	How do you evaluate build-vs-buy for infrastructure tooling?
8	Architect a self-service infrastructure platform
9	How do you handle a 100-service platform migration?
10	Design an SRE practice from scratch
11-50	*(C-level communication of technical strategy, budget management for infrastructure, vendor negotiation, staffing and team structure, cross-functional alignment, innovation labs, tech debt payoff strategy, engineering excellence metrics, developer productivity measurement, platform team OKRs, incident command system, regulated industry DevOps, acquisition integration, legacy modernization roadmap, edge*

#	Question
	computing strategy, AI/ML infrastructure, sustainability in cloud, open-source contribution strategy, conference speaking preparation, technical writing and documentation leadership)

> ***You've completed the full handbook. Want me to generate an even deeper version focused specifically on Azure DevOps, Terraform, Kubernetes, and consulting-style architecture questions?***